Implementation and Verification of a CAN-FD Bus Transceiver in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

Abstract

*This document is a work-in-progress draft for the B.Eng thesis. Currently, the project is in the **Verification Plan** phase (Phase 2), with architectural prototyping for the transmitter sub-system completed.*

This thesis describes the design, implementation, and verification of a CAN (Controller Area Network) and CAN-FD (Flexible Data rate) transmitter sub-system. The implementation is targeting high-reliability engine controller applications and is compliant with the ISO 11898-1:2024 standard [1]. Key features include support for both Classic and FD frame formats, Transmitter Delay Compensation (TDC) for high-speed data phases, and a modular architecture separated into Link Layer Control (LLC), Media Access Control (MAC), and Physical Coding Sublayer (PCS).

Abbreviations

Table 1: Abbreviations used in this report.

Abbreviation	Meaning
ACK	Acknowledgement
AF	Acceptance Field
AUI	Attachment Unit Interface
CAN	Controller Area Network
CBFF	Classical Base Frame Format
CEFF	Classical Extended Frame Format
DF	Data Frame
DLL	Data Link Layer
EF	Error Frame
FBFF	FD Base Frame Format
FCE	Fault Confinement Entity
FEFF	FD Extended Frame Format
FTYP	Frame Type
LLC	Logical Link Control
LSDU	LLC Service Data Unit
MAC	Medium Access Control
OF	Overload Frame
OSI	Open Systems Interconnection
PCS	Physical Coding Sublayer
PDU	Protocol Data Unit
PMA	Physical Medium Attachment
RF	Remote Frame
SAP	Service Access Point
SDU	Service Data Unit
SOF	Start of Frame
TEC/REC	Transmit Error Counter / Receive Error Counter
VCID	Virtual CAN Channel Identifier

1 Introduction

TODO: Add a section on the IO extender board (The board is on the test wall, get the name from Alex)

1.1 Motivation

The Controller Area Network (CAN) has been the workhorse of automotive and industrial communication for decades. However, the increasing bandwidth requirements of modern systems led to the development of CAN-FD (Flexible Data rate), which allows for larger payloads and higher bit rates. This project aims to provide a robust, hardware-independent VHDL implementation of a CAN-FD transmitter.

1.2 Existing CAN Controller

The starting point for this project is an existing CAN Classic controller developed internally at the company. The controller is implemented in VHDL and has been integrated into a production IO-extender FPGA design. It supports CAN Classic frames with both 11-bit (base) and 29-bit (extended) identifiers at bit rates up to 500 kbit/s, and has been verified through hardware bring-up on physical CAN buses.

The controller follows a monolithic architecture where a single top-level wrapper (`can_bus_controller`) instantiates six sub-modules: a combined TX/RX frame FSM (`can_fsm`), a bit timing generator (`can_node_clock`), a dynamic bit stuffer (`can_stuff_bit_gen`), a CRC-15 engine (`gen_crc`), and two Avalon-ST converters for frame serialization and deserialization (`can_ast_to_serial`, `can_serial_to_ast`). The entire design totals roughly 2,000 lines of VHDL across seven source files.

The central component is `can_fsm`, an 810-line state machine with 18 states that handles both transmission and reception in a single process. The FSM manages frame arbitration, bit-level transmission and reception, stuff bit error checking, CRC validation, ACK handling, and error flag generation. Error counting (TEC/REC) and node state transitions (Error Active, Error Passive, Bus Off) are handled in separate processes within the same file but are tightly coupled to the main FSM through shared signals.

1.2.1 Limitations

While the existing controller is functional for CAN Classic, several architectural limitations prevent it from being extended to support CAN FD:

Single bit rate domain. The `can_node_clock` module generates a single pair of timing strobes - one sample pulse and one transmit pulse - derived from a fixed set of bit timing parameters. CAN FD requires switching between a nominal bit rate (used during arbitration) and a faster data bit rate (used during the data phase), with Transmitter Delay Compensation (TDC) to account for the transceiver round-trip delay at the higher rate. Adding dual bit rate support and TDC to the existing clock module would require a fundamental redesign of the timing architecture.

Dynamic bit stuffing only. The `can_stuff_bit_gen` module implements the CAN Classic rule of inserting an inverse bit after five consecutive identical bits. CAN FD introduces a second stuffing mode - fixed bit stuffing - where a stuff bit is inserted at fixed intervals during the CRC field, and a Stuff Bit Count (SBC) field with Gray-coded parity is appended. The existing stuffer has no mechanism for mode switching or SBC generation.

Single CRC polynomial. The controller uses a single CRC-15 instance. CAN FD requires three CRC polynomials: CRC-15 for Classic frames, CRC-17 for FD frames with payloads up to 16 bytes, and CRC-21 for

larger FD payloads. Furthermore, the CRC data feed differs between Classic and FD - in FD frames, dynamic stuff bits in the arbitration region are included in the CRC computation, requiring a dual data feed to the CRC engine.

Combined TX/RX FSM. The monolithic FSM interleaves transmission and reception logic in every state, with the `is_transmitter` flag selecting the active code path. This coupling makes it difficult to add FD-specific states (such as BRS, ESI, and the SBC field) without increasing the already high cyclomatic complexity. A CAN FD frame has more control fields than a Classic frame, and handling both TX and RX paths for all four frame formats (Classic Base, Classic Extended, FD Base, FD Extended) in a single process would result in an unwieldy state machine.

Embedded error handling. Error detection, error flag transmission, and error counter management are distributed across the main FSM process and its auxiliary processes. The ISO 11898-1 standard defines the Fault Confinement Entity (FCE) as a logically separate component with well-defined interfaces to the MAC and PCS sub-layers. Extracting the error handling into a reusable, independently testable FCE module - as required by the standard's layered architecture - would require significant refactoring of the existing FSM.

1.2.2 Decision to Redesign

Given these limitations, extending the existing controller to CAN FD would require modifying nearly every sub-module and fundamentally restructuring the FSM. The resulting design would carry the constraints of the original monolithic architecture while trying to support a significantly more complex protocol. Instead, a clean-sheet redesign was chosen, structured around the ISO 11898-1 layered architecture (LLC, MAC, PCS, FCE) with separated TX and RX paths, reusable sub-components (bit stuffer, CRC engine), and typed record interfaces. This approach allows each sub-layer to be implemented and verified independently, supports both Classic and FD frame formats from the outset, and produces a modular design that can be maintained and extended without cross-cutting changes.

1.3 Existing CAN FD IP Cores

Before committing to an in-house redesign, the available CAN FD controller IP cores were evaluated. Table 2 summarizes the candidates, spanning both open-source and commercial offerings.

1.3.1 Open-Source Implementations

CTU CAN FD [2], [3] is the only mature open-source CAN FD controller available as synthesizable HDL. Developed at the Czech Technical University in Prague, it is written in VHDL, licensed under MIT, and has been conformance-tested against ISO 16845-1 [4]. The controller includes a full TX and RX pipeline with up to four TX buffers, acceptance filtering, timestamping, and a register interface with DMA support. A mainline Linux kernel driver has been available since kernel version 5.12. CTU CAN FD represents a complete, production-oriented CAN node - a significantly broader scope than what is needed in this project.

OpenCores CAN [5] is a Verilog controller modeled after the Philips SJA1000 register interface. It is one of the earliest open-source CAN cores and is widely cited in academic work. However, it supports only CAN 2.0B (Classic CAN) and has seen no active development since approximately 2010. It cannot serve as a starting point for CAN FD.

Canola [6] is a VHDL CAN 2.0B controller with a clean VHDL-2008 codebase and a cocotb-based testbench. It includes a Triple Modular Redundancy (TMR) wrapper for radiation-tolerant applications. Like the OpenCores core, it does not support CAN FD.

1.3.2 Commercial Implementations

Bosch M_CAN [7], [8] is the reference CAN FD controller, developed by the inventor of both CAN and CAN FD. M_CAN is the IP core embedded in virtually every automotive microcontroller (NXP S32, Infineon AURIX, STM32, TI Jacinto, Renesas RH850). It is licensed under NDA with per-design royalty fees.

AMD/Xilinx CAN FD [9] is a soft IP core included in the Vivado Design Suite. It provides an AXI4-Lite register interface with up to 32 acceptance filters, TX mailboxes, and RX FIFOs. It is device-locked to AMD/Xilinx FPGAs and cannot be ported to other targets.

CAST CAN FD [10] is a technology-independent RTL core with AMBA APB/AHB bus interface options. It is licensed per-design with an upfront fee. Synopsys (DesignWare) and Cadence offer similar ASIC-targeted CAN FD cores under their respective IP licensing programs.

Table 2: Survey of available CAN FD controller IP cores.

Implementation	Language	CAN FD	License	Scope	Conformance Tested
CTU CAN FD [2]	VHDL	Yes	MIT	Full node (TX+RX, buffers, DMA)	ISO 16845-1
OpenCores CAN [5]	Verilog	No	LGPL	Full node (CAN 2.0B only)	No
Canola [6]	VHDL	No	MIT	Full node (CAN 2.0B, TMR)	No
Bosch M_CAN [7]	HDL (NDA)	Yes	Per-design royalty	Full node	Yes (reference)
AMD/Xilinx CAN FD [9]	HDL	Yes	Vivado-included	Full node	Yes
CAST CAN FD [10]	RTL	Yes	Per-design fee	Full node	Yes

1.3.3 Rationale for In-House Development

Despite the availability of these solutions, none satisfies the combined requirements of a large engine design company developing safety-critical control systems. The decision to develop the CAN FD transceiver in-house was driven by five considerations.

TODO: Add something about the 30 year maintenance requirements that the company is responsible for (hard to rely on 3rd party products for this long). **IP ownership and supply chain independence.** Commercial IP cores introduce a licensing dependency on an external vendor. For a product with a multi-decade lifecycle - typical in heavy-duty engine applications - this dependency carries risk: vendors may discontinue support, change licensing terms, or be acquired. Owning the RTL outright eliminates this exposure and ensures that the design can be maintained, ported, and modified without third-party approval for the full product lifetime. The open-source CTU CAN FD avoids the licensing risk, but using it still means adopting a codebase whose architecture, naming conventions, and design decisions were made for a different context.

Verification authority. In safety-critical domains, the verification evidence must be traceable from standard requirements to RTL assertions and testbench results. Adopting a third-party core - even one

conformance-tested against ISO 16845-1 - means inheriting its verification artifacts rather than producing them. The company's verification methodology requires full control over the verification plan, the testbench architecture, and the assertion coverage. Building the RTL in-house allows the verification plan (described in Section ??) to drive the implementation, ensuring that every module is verified against the specific requirements extracted from the standard, using the company's own toolchain and conventions.

Architectural scope. All available IP cores implement a complete CAN node: TX and RX pipelines, message RAM, acceptance filtering, buffer management, register interfaces, and in some cases DMA controllers. The company's application requires only the data link layer (LLC, MAC, FCE) and physical coding sublayer (PCS) - the protocol engine that converts between byte-level frame data and the serial bus. The higher-level buffering and filtering logic already exists in the company's FPGA infrastructure. Adopting a full-node IP core would introduce unnecessary complexity and area overhead, and the integration effort to bypass or disable the unused subsystems may approach the effort of a targeted implementation.

Integration with existing infrastructure. The company's FPGA designs use a specific Avalon-ST streaming interface for inter-module communication, a particular clock and reset architecture, and established conventions for signal naming and module boundaries. A third-party core would require an adaptation layer to bridge its native interface (AXI, APB, or custom register map) to the existing infrastructure. The in-house design uses the company's interface conventions natively, eliminating this integration overhead.

Platform independence. The AMD/Xilinx CAN FD core is locked to Xilinx devices. The Bosch M_CAN and other commercial cores are delivered as technology-specific netlists or encrypted RTL for a particular target. The in-house design is written in portable VHDL-2008, synthesizable on any FPGA platform or ASIC flow, ensuring that the IP remains usable if the company changes FPGA vendors.

1.4 Problem Statement

The need for CAN FD support in the company's engine controller platform, combined with the architectural limitations of the existing CAN Classic controller (Section 1.2.1) and the unsuitability of available third-party IP cores for the company's specific requirements (Section 1.3.3), motivates the development of a new CAN FD transceiver from the ground up. The challenge lies in creating a modular, independently verifiable implementation that supports both Classic and FD frame formats, handles dual bit rate switching with Transmitter Delay Compensation (TDC), and maintains strict compliance with ISO 11898-1 [1] - all while fitting into the company's existing FPGA infrastructure and verification methodology.

1.5 Objectives

- Implement a VHDL-2008 compliant CAN/CAN-FD transmitter.
- Support both Base (11-bit) and Extended (29-bit) identifiers.
- Implement TDC measurement and compensation logic.
- Ensure high verification coverage using OSVVM and GHDL.

2 Background

2.1 CAN as a Communication Bus

The Controller Area Network (CAN) is a serial communication bus developed by Bosch in 1986 **?** to connect electronic control units (ECUs) in automotive environments without a central host computer. Where point-to-point wiring and star-switched architectures require a dedicated conductor between every com-

municating pair, CAN uses a shared two-wire differential bus on which all nodes broadcast simultaneously and arbitrate access without any designated bus master. Any node may initiate a transmission at any time; contention is resolved by a non-destructive bitwise arbitration in which the transmitter with the lower-priority identifier detects the collision and silently withdraws, leaving the winner's frame intact. Differential signaling on a twisted pair (ISO 11898-2 physical layer) provides strong common-mode noise rejection - a practical necessity in the electrically harsh environment of an engine bay or industrial cabinet.

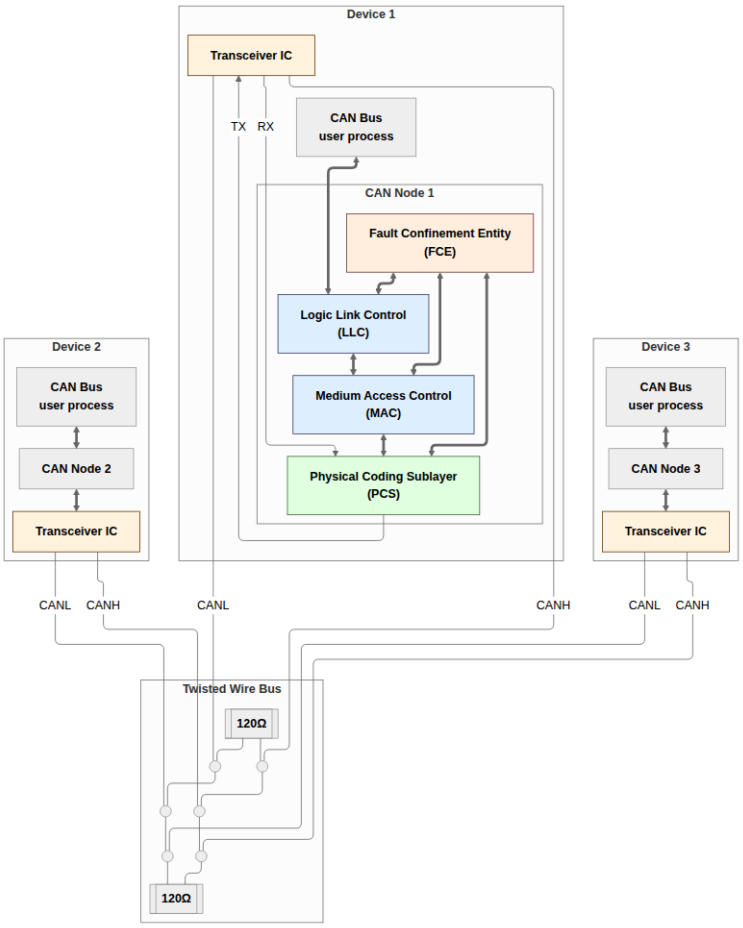

Figure 1: CAN bus consisting of three CAN nodes connected via a shared differential two-wire bus. Each node contains a CAN controller and transceiver; termination resistors at each end of the bus prevent signal reflections.

CAN's error-handling architecture is a distinguishing feature relative to simpler serial protocols. Five complementary error detection mechanisms operate concurrently on every transmitted frame: bit monitoring, frame check, cyclic redundancy checking, acknowledgement checking, and bit stuffing violation detection. A fault confinement mechanism tracks each node's error history and automatically escalates from error-active operation through error-passive to a bus-off state in which a persistently faulty node is electrically isolated from the network, preventing it from corrupting communication between healthy nodes. Together these properties made CAN the protocol of choice for safety-relevant in-vehicle networks; adoption subsequently spread to industrial automation, medical devices, and aerospace ground support equipment.

CAN's original data payload was capped at eight bytes per frame, limiting raw throughput to around 1 Mbit/s. As embedded control applications became more data-intensive, this ceiling became a practical constraint. CAN FD (Flexible Data-Rate), finalized by Bosch in 2012 and incorporated into ISO 11898-1 in 2015, extends the maximum payload to 64 bytes and introduces a separate higher-speed data phase

with bit rates up to 8 Mbit/s or beyond, while preserving the Classic CAN arbitration phase and the fault-confinement architecture unchanged. The governing standard for this project is ISO 11898-1:2015, which specifies both Classic CAN and CAN FD data link layer and physical signaling requirements.

2.2 VHDL-2008 and OSVVM

The implementation language for this project is VHDL-2008, the current revision of the IEEE VHDL standard. VHDL-2008 adds several features relevant to parameterized hardware design and verification: unconstrained record elements, enhanced generic lists, and improved support for the IEEE numeric packages. The GHDL open-source simulator [11] supports VHDL-2008 natively and is used for all simulation in this project.

The verification framework is OSVVM (Open Source VHDL Verification Methodology) [12], a VHDL-native library providing test infrastructure including clock and reset generation, constrained-random stimulus, functional coverage, and a uniform pass/fail reporting framework. OSVVM procedures replace ad-hoc signal manipulation in testbenches, ensuring that timing relationships are expressed in terms of clock cycles rather than time literals and that pass/fail decisions are logged uniformly across all test scenarios.

3 Requirements

Bridging the gap between a normative specification document and a structured, verifiable requirements set is a pivotal task in any protocol implementation project. The present section describes the process of distilling a coherent set of verifiable requirements from the ISO 11898-1 standard. In addition, the constraints imposed by general company practices and standards are described along with project-specific requirements related to the larger system in which the module is intended to integrate.

3.1 VHDL Code Standard and Design Constraints

The constraints on this project come from two distinct sources. Two requirements are specific to this project's place within the company's existing CAN infrastructure while the remaining constraints come from the company's VHDL Code Standard and apply uniformly to all FPGA IP modules developed in-house.

3.1.1 Project-specific Infrastructure Requirements

1. **Avalon-ST user interface:** The CAN controller's external interface to the host system must use the Avalon-ST streaming protocol [13] (data, valid, ready, sop, eop). The requirement applies specifically to the boundary between the CAN controller and its user.
2. **IP library CRC block:** The company maintains a reusable, parameterised CRC generator (`gen_crc`) in its IP library. This module must be used for all CRC computations.

3.1.2 File Structure and Naming

1. **Per-module file structure:** Each module must be organized into a fixed directory layout: `hdl_src/` for RTL source files, `hdl_tb/` for test bench files, and `test_case/` for waveform configuration. One entity per VHDL file, with the filename matching the entity name. Every entity requires a dedicated test bench, unless it is instantiated exclusively as a sub-module of a fully tested parent.
2. **Entity port types:** Entity ports are restricted to `std_logic`, `std_logic_vector`, and records or arrays of these types. Modes are restricted to `in` and `out`.

3. **Naming conventions:** A mandatory prefix/suffix scheme applies to all VHDL identifiers: types (`t_`), constants (`c_`), generics (`gc_`), processes (`p_`), functions (`f_`), packages (`pk_`), state variables (`s_`), entity inputs (`_i`), and entity outputs (`_o`).

3.1.3 RTL Coding Style and Verification

1. **RTL design rules:** Synchronous processes must be sensitive to the clock only. Reset must be synchronous and initialize all control registers. FSMs are preferably implemented as single-process designs where all signal assignments are derived from the current state, with an explicit other-state that returns to a known safe state.
2. **Test bench requirements:** Test benches must follow a black-box testing model and test cases must be ordered as: reset tests first, then a normal-usage test, then all remaining tests.

3.2 From Specification to Structured Requirements

The requirements engineering process was aimed at tackling two key objectives:

1. Extracting a clear and actionable set of requirements that could serve as a starting point for the design phase.
2. Establishing a clear, traceable link between the ISO 11898-1 specification and the verification environment.

Both objectives are complicated by the nature of the source material. Normative requirements are distributed across subsections, often restated from different perspectives, and interspersed with explanatory and descriptive text. Accordingly, extracting a precise unambiguous set of requirements from such prose is non-trivial and inherently prone to oversights and misinterpretations. Nonetheless, a structured and precise requirements set is absolutely essential. It serves as both the starting point for the system design phase and as the target of verification effort. [14]

The requirements set was constructed using the AI-assisted pipeline shown in Figure 2. The first step was converting the ISO 11898-1 pdf to Markdown - a format which can be efficiently searched and ingested by LLM models. The resulting Markdown file was then fed to a Claude Sonnet 4.6 LLM agent, which was prompted to extract all normative statements - sentences containing words like “shall”, “should”, “must”, and their corresponding negations.

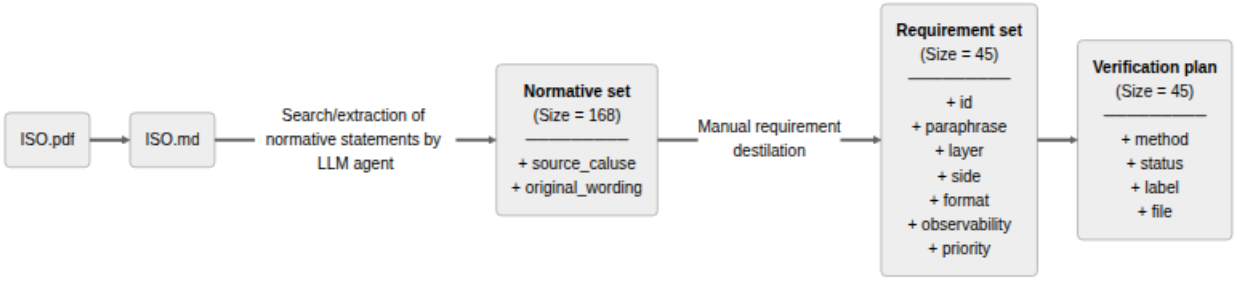

Figure 2: Pipeline generating the `verification_plan.toml` artifact. 1) LLM agent extraction of normative statements from the ISO 11898-1 standard. 2) Manual grouping of related normative statements and requirement distillation. 3) Argumentation with additional requirement labels and verification specific fields.

This process yielded a raw set of 168 normative statements linked to the ISO standard sections from which they were extracted. The normative set was then manually reviewed, consolidated, and distilled into a final

set of 45 requirements (reproduced in Section A).

3.3 AI-Assisted Extraction: Utility and Limitations

The LLM agent earned its keep by bootstrapping and linking the initial normative statement set. Having a fully populated and linked starting point - even one requiring substantial revision - gave the manual review process a concrete artifact to work from. That said, the overall time saving was likely marginal. The agent's output had to be reviewed statement by statement, which is functionally similar to extracting requirements manually in the first place. The primary benefit of the AI-assisted approach was therefore not efficiency, but rather the increased consistency associated with an automated extraction process.

The 45-requirement set establishes the scope of the verification effort. Before the classification dimensions that structure the verification plan can be introduced, the following section presents the protocol in the detail those dimensions presuppose: the sub-layer model that the layer dimension maps onto, the four frame formats the format dimension enumerates, the bit timing and dual-rate concepts that underlie several P1 requirements, and the error-handling model that drives the FCE module separation.

4 CAN and CAN-FD Protocol Overview

This section covers the ISO 11898-1 layered reference model, frame types and fields, bit timing and the dual-rate mechanism that distinguishes CAN FD from Classic CAN, bit stuffing, CRC, and error handling. A reader already familiar with ISO 11898-1 may skip to Section 5.

4.1 Layered Reference Model

ISO 11898-1 structures the CAN data link layer into three functional sub-layers and a cross-cutting Fault Confinement Entity (FCE):

- **LLC (Logical Link Control)**: accepts frame requests from the host application, applies retransmission policy on error or lost arbitration, and supplies frames to the MAC in serialized form.
- **MAC (Medium Access Control)**: encodes and decodes the frame bit-by-bit - performing bit stuffing and destuffing, CRC generation and checking, and acknowledgement handling - and governs bus access arbitration.
- **PCS (Physical Coding Sub-layer)**: manages bit timing, clock synchronization (including Transmitter Delay Compensation for FD data phase), and the sample/drive interface to the physical transceiver.
- **FCE (Fault Confinement Entity)**: maintains Transmit Error Counter (TEC) and Receive Error Counter (REC), escalating the node's error state from Error Active through Error Passive to Bus Off as error counts accumulate.

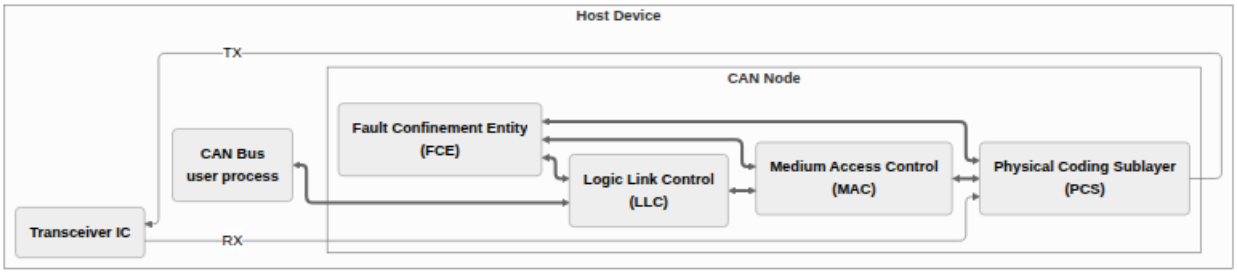

Figure 3: CAN node.

In the implementation described in this report, each sub-layer maps to a dedicated VHDL module, and the sub-layer interfaces become the port records connecting those modules (Section 6).

4.2 Frame Types and Formats

CAN defines two classes of frames: Classic CAN (CC) and CAN FD (FD). Within each class, frames may carry either an 11-bit base identifier or a 29-bit extended identifier, giving four in-scope formats (Table 3). Remote frames and CAN XL frames are out of scope for this project.

Table 3: In-scope frame formats and their key parameters.

Format	Identifier	Max payload	CRC	Bit stuffing
CB (Classic Base)	11-bit	8 bytes	CRC-15	Dynamic (5-in-a-row)
CE (Classic Extended)	29-bit	8 bytes	CRC-15	Dynamic (5-in-a-row)
FB (FD Base)	11-bit	64 bytes	CRC-17 or CRC-21	Dynamic + fixed
FE (FD Extended)	29-bit	64 bytes	CRC-17 or CRC-21	Dynamic + fixed

A Classic CAN frame consists of: Start of Frame (SOF), Arbitration field (identifier, RTR, IDE), Control field (DLC), Data field, CRC field, ACK slot and delimiter, End of Frame, and Intermission. A CAN FD frame shares the same structure through the arbitration phase, then introduces the FDF bit (marking the frame as FD), followed by BRS (Bit Rate Switch) and ESI (Error State Indicator) control bits that govern the transition into the higher-speed data phase.

Table 4: Field width and bit stuffing encoding for multi-bit fields in CB, CE, FB and FE frames.

Format	Identifier	Width	Bit stuffing
CB, CE, FB, FE	IDB	11 Bits	Dynamic
CE, FE	IEXT	18 Bits	Dynamic
CB, CE, FB, FE	DLC	4 Bits	Dynamic
FB, FE	DATA	0-64 Bytes	Dynamic
CB, CE	DATA	0-8 Bytes	Dynamic
FB, FE	CRC	17 Bits (DLC <= 16), 21 Bits (DLC > 16)	Fixed
CB, CE	CRC	15 Bits	Dynamic
FB, FE	SBC	4 Bits	Fixed

4.3 Bit Timing and Flexible Data Rate

Every CAN bit period is divided into four non-overlapping time segments measured in Time Quanta (TQ), where one TQ equals the period of the prescaled system clock:

- **Sync Segment (SYNC_SEG)**: one TQ; the point at which the bus is expected to produce a recessive-to-dominant edge after synchronization.
- **Propagation Segment (PROP_SEG)**: compensates for round-trip signal propagation delay on the bus and in the transceiver; configurable.

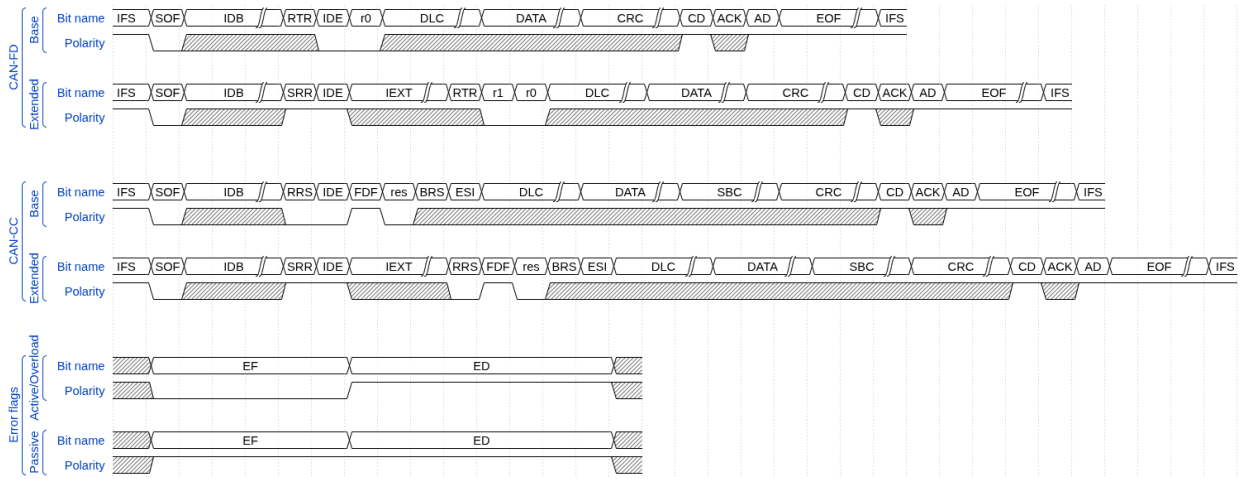

Figure 4: Frame and error flag formats for CAN-CC Base, CAN-CC Extended, CAN-DF Base and CAN-DF Extended. Widths of truncated multi-bit fields are tabulated in Table 4.

- **Phase Segment 1 (PHASE_SEG1)**: immediately precedes the sample point; can be lengthened by the resynchronization mechanism to absorb positive phase errors.
- **Phase Segment 2 (PHASE_SEG2)**: follows the sample point to the end of the bit; can be shortened to absorb negative phase errors.

The **sample point** falls at the PHASE_SEG1 / PHASE_SEG2 boundary. Every receiver samples the bus exactly once per bit at this point; the sampled polarity is the received bit value. The sample point position - expressed as a percentage of the total bit time - is a configuration parameter traded off against bus length, node count, and oscillator tolerance.

Resynchronization corrects for accumulated phase error between a receiver's local oscillator and the transmitter's bus edges. Hard synchronization forces a full re-alignment on the SOF falling edge at the start of each frame. During the frame, soft resynchronization adjusts PHASE_SEG1 or PHASE_SEG2 by up to the configured Synchronization Jump Width (SJW) on each recessive-to-dominant edge, keeping the sample point aligned with the transmitter.

CAN FD and the flexible data rate. CAN FD introduces a second, independently configured bit rate for the data phase. The BRS (Bit Rate Switch) bit in the FD control field (Table 3) controls this transition: when BRS is recessive, the bus switches to the data-phase bit rate immediately after the BRS sample point and returns to the nominal rate at the CRC delimiter. The nominal rate governs the arbitration phase (SOF through BRS) and the return path (CRC delimiter onward); the data rate governs the payload and CRC fields in between. Because the data phase operates at a much shorter bit time, the same physical propagation delay represents a larger fraction of the bit period. On electrically long buses at high data rates, the loop propagation delay can exceed a full data-phase bit time.

Transmitter Delay Compensation (TDC) addresses this. A transmitter in the FD data phase cannot rely on immediate bus loopback for bit-error monitoring, because the echo of a driven bit arrives one or more bit times late. TDC measures the actual round-trip delay at the start of the data phase and configures a Secondary Sample Point (SSP) at the correct offset, so that each transmitted bit is still checked for loopback correctness. The TDC measurement and SSP configuration are PCS responsibilities and are a significant driver of PCS complexity in the implementation (Section **??**).

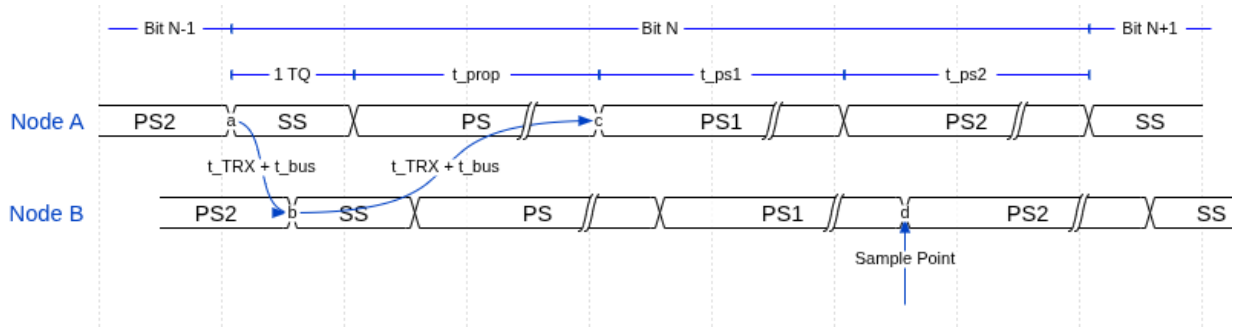

Figure 5: CAN bit time structure. One bit consists of SYNC_SEG (SS) which is 1 Time Quantum (TQ) long, PROP_SEG (PS), PHASE_SEG1 (PS1), and PHASE_SEG2 (PS2). The sample point (SP) sits at the PHASE_SEG1/PHASE_SEG2 boundary. The figure illustrates a to-node synchronised bus with bus wire delay t_bus and transceiver delay t_TRX.

4.4 Bit Stuffing

Bit stuffing ensures sufficient transitions on the bus for receiver clock synchronization. Classic CAN applies dynamic stuffing throughout the frame: after five consecutive bits of the same polarity, the transmitter inserts one complement stuff bit and the receiver removes it before forwarding the data stream. CAN FD retains dynamic stuffing through the arbitration phase, then switches to a combined dynamic-plus-fixed scheme in the data phase. Fixed stuff bits are inserted at predetermined positions (every fourth bit in the CRC field, independent of the preceding bit pattern); they carry a parity-encoded Stuff Bit Count (SBC) field that allows receivers to independently verify the number of dynamic stuff bits seen in the frame - an additional error detection layer absent in Classic CAN.

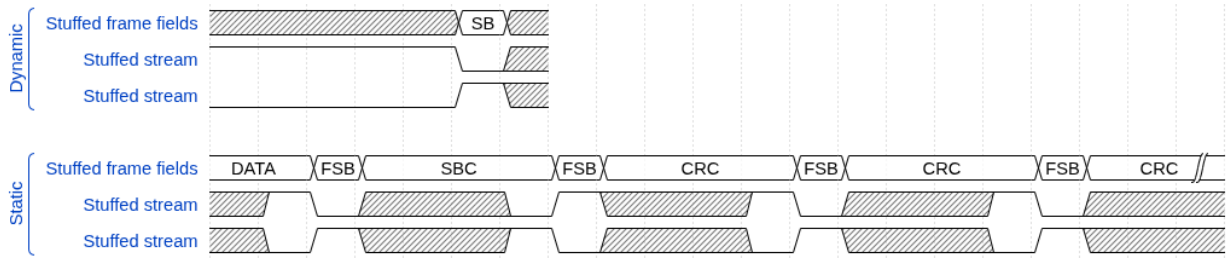

Figure 6: Dynamically and statically bit-stuffed stream examples. In frame fields encoded with dynamic bit-stuffing an opposite polarity stuff bit (SB) is inserted after five consecutive same-polarity bits. In frame fields encoded with static bit-stuffing (SBC and CRC in FD frames) a fixed stuff bit (FSB) is inserted after each fourth bit. A FSB is also inserted before the first bit of the SBC field.

4.5 Cyclic Redundancy Check

The CRC polynomial and length depend on frame format and data length:

- **CRC-15**: used for all Classic CAN frames; covers the stuffed bit stream from SOF through the data field.
- **CRC-17**: used for FD frames with data lengths up to 16 bytes.
- **CRC-21**: used for FD frames with data lengths from 20 to 64 bytes.

For FD frames, the CRC is computed over the fixed-stuff-bit-augmented bit stream rather than the raw data, meaning CRC computation must be interleaved with stuff bit insertion. A received frame with a CRC mismatch triggers an Error Flag.

4.6 Error Detection and Fault Confinement

Every CAN node monitors the bus for five categories of error. Bit errors occur when a transmitter reads back a polarity different from what it drove. Stuff errors occur when six consecutive bits of the same polarity appear where the stuffing rule prohibits it. CRC errors occur when the received checksum does not match the locally recomputed value. Form errors occur when fixed-format fields contain illegal bit values. Acknowledgement errors occur when a transmitter receives no dominant ACK bit from any receiver. Detection of any error causes the detecting node to immediately transmit an Error Flag, aborting the in-progress frame.

The FCE tracks each node's error history through TEC and REC. Counter increments and decrements follow the rules in ISO 11898-1 Section 12. A node begins in Error Active and transitions to Error Passive when either counter exceeds 127, then to Bus Off when TEC exceeds 255. In Bus Off the node ceases all bus activity until 128 sequences of 11 consecutive recessive bits are observed, after which it returns to Error Active. This escalation mechanism is the subject of several verification plan requirements and directly motivates the separation of the FCE into a dedicated module with its own testbench.

With those mechanisms established - sub-layer boundaries, frame formats, bit timing and the dual data rate, stuffing rules, CRC polynomials, and the fault confinement escalation model - the following section introduces the five classification dimensions of the verification plan and shows how each one connects back to the protocol concepts described here.

5 Verification Plan

The 45 requirements from Section 3 were each classified along five dimensions. Each dimension draws directly on the protocol concepts presented in Section 4: the layer dimension maps onto the sub-layer model (Section 4.1); the format dimension maps onto the four frame types of Table 3; the observability dimension reflects the distinction between externally visible behavior and internal protocol state that the error-model and CRC sections illustrate concretely. Together the dimensions make the required testbench configuration and evidence type explicit for each requirement, without leaving either to be inferred from the requirement text alone. The dimensions are:

- layer, Section 5.1
- side, Section 5.2
- format, Section 5.3
- observability, Section 5.4
- priority, Section 5.5

The following subsections describe the rationale and allowed values for each dimension, followed by the full verification plan data structure and its traceability fields.

5.1 Layer

The layer field assigns each requirement to the protocol sub-layer that owns it (LLC, MAC, PCS, or FCE - see Section 4.1), determining the verification boundary at which the requirement must be exercised. A fifth label - **system** - classifies requirements that are inherently multi-layer or multi-node in character. Some CAN behaviors cannot be attributed to a single layer of a single node: they emerge from interactions between multiple nodes on the bus, or span the layer boundary within a single node. The system label flags these requirements as ones that require either an integrated multi-module testbench or a multi-node

simulation environment.

At design time, this classification directly motivated the layered module architecture: requirements assigned to a given layer pointed to the corresponding module as the responsible implementation unit and to that module's testbench as the primary verification environment. The consequence of the system label is described further in Section 6.2.2.

5.2 Side

The side field records whether a requirement pertains to the transmitter path, the receiver path, or both roles simultaneously. This dimension reflects the ISO standard's own framing, which frequently specifies transmitter and receiver obligations separately. In the verification environment, the side field determines whether a testbench drives the DUT in transmitter mode, receiver mode, or both roles in succession within a single test scenario. The design consequences of this dimension - and why it appeared to motivate a split-path architecture but did not - are discussed in Section 6.2.2.

5.3 Format Applicability

The format_applicability field records which of the four in-scope frame formats (CB, CE, FB, FE - see Table 3) each requirement applies to. Because the formats differ in stuffing mode, CRC polynomial, and control field structure (Section 4), a requirement that applies only to FD frames implies stimulus configurations with FDF=1 and DLC values spanning both the CRC-17 and CRC-21 threshold, while a requirement that applies to all four formats must be exercised across all format-specific configurations. The field makes those implications explicit rather than leaving them to be inferred from the requirement text.

5.4 Observability

The observability field resolves each requirement as either black-box or white-box, relative to the module boundary of the owning layer:

- **Black-box**: Can be verified purely through the module's observable port signals.
- **White-box**: Verification requires direct observation of the module's internal state.

This distinction has direct consequences for testbench architecture. Black-box requirements are verifiable with stimulus-and-observe testbenches that drive inputs and check outputs without any knowledge of internal implementation. White-box requirements - which include CRC polynomial correctness, bit counter arithmetic, error counter thresholds, and Gray-coded SBC encoding - require either PSL assertions on internal signals or a parallel reference model that re-computes the expected value independently. In the verification plan, white-box requirements are the primary driver for embedding PSL assertions directly in the RTL source files, where they have access to internal signals regardless of module hierarchy.

5.5 Priority

The priority field classifies each requirement into one of three levels:

- P1 requirements are need-to-have - they must be verified before the design can be considered complete.
- P2 requirements are nice-to-have - they are verified in the normal verification cycle but do not block closure.
- P3 requirements are optional - addressed only if schedule permits.

The final plan contains 31 P1, 13 P2, and 1 P3 requirements. The single P3 requirement covers optional transmitter delay compensation accuracy bounds that go beyond the ISO minimum; it was deferred because the core TDC mechanism is covered by P1 requirements and the accuracy bound can only be assessed against a hardware target.

5.6 Verification Plan Data Structure

The verification plan data structure (Table 5) augments each requirement with the dimensions needed to answer not just *what* must be true, but *how* it will be verified, *where* the evidence lives, and *when* verification is complete. The plan evolved continuously throughout the project as implementation and verification work progressed. The following sub-sections detail the rationale and allowed values for each field in the verification plan. The complete verification plan is reproduced in Section A as two separate tables (linked by common ID's).

Table 5: Verification-plan data structure fields.

Field	Purpose
`id`	Sequential identifier REQ-NNN.
`source_clause`	ISO 11898-1:2024 section reference.
`original_wording`	Verbatim normative text excerpets from the ISO standard.
`paraphrase`	Concise paraphrase of the `original_wording` field (this is the actual requirement)
`layer`	Sub-layer owner: LLC, MAC, PCS, FCE, or system (Section 5.1).
`side`	transmitter, receiver, or both (Section 5.2).
`format_applicability`	Applicable frame formats: CB, CE, FB, FE (Section 5.3).
`observability`	`black_box` or `white_box` (Section 5.4).
`verification_method`	Method(s) used to verify the requirement (Section 5.6.1).
`priority`	P1 (need-to-have), P2 (nice-to-have), or P3 (optional) (Section 5.5).
`status`	`not_started`, `in_progress` or `complete` (Section 5.6.3).
`notes`	The field is intended to clarify residual ambiguity left over from the other fields
`label`	Assertion label, TB procedure name, coverage ID, or RTL tag. Comma-separated when multiple procedures cover distinct sub-claims (Section 5.6.2).
`file`	Target file: TB for simulation/coverage, RTL for code inspection. Comma-separated when sub-claims span multiple files (Section 5.6.2).

5.6.1 Verification Method

The verification_method field makes the path from requirement to verification artifact explicit and actionable. Four methods are used: `simulation` (automated assertion procedures in a test bench), `code_inspection` (RTL source review), `waveform_inspection` (manual review of simulation output),

and `coverage` (coverage bins a value range). Combinations are valid when multiple sub-claims within one requirement each call for a different method.

5.6.2 Traceability: Label and File

Each requirement entry carries two dedicated traceability fields: a `file` field identifying the test bench or RTL source file responsible for covering the requirement, and a `label` field identifying a specific named procedure, assertion, or coverage ID within that file. Together they establish a direct, navigable link from each requirement to its verification artifact.

5.6.3 Status

The status field (`not_started`, `in_progress`, `complete`) records requirement closure state explicitly, allowing partial progress to be tracked.

The verification plan - 45 requirements each classified by layer, side, format, observability, and priority, and each linked to a testbench file and assertion label - constituted the principal set of architectural inputs for the design phase. How those inputs shaped the module decomposition, and where the apparent mapping from requirements structure to design structure broke down, is the subject of the next section.

6 Design and Architecture

6.1 Ramifications of the Requirements Model on Initial Design Strategy

The structure of the requirements model had direct consequences for the initial design strategy, in ways that were not fully anticipated at the outset.

The **layer dimension** mapped naturally onto the ISO standard's own layered reference model, making a layered module architecture look like the obvious and well-motivated implementation strategy. A dedicated hardware module for each layer - MAC, LLC, fault confinement, and PCS - would allow requirements pertaining to a given layer to be verified in isolation against that layer's module, rather than through the surface of a fully integrated system where internal behavior is obscured by surrounding logic. The observability dimension reinforced this directly: black-box requirements mapped cleanly onto port-level stimulus and observation, while white-box requirements pointed toward the need for reference models or PSL assertions on internal signals, both of which are most tractable in a per-module testbench. In retrospect, this was a sound conclusion. The modular architecture proved to be the right design choice, and the requirements table provided a well-motivated rationale for it from the start.

The **TX/RX side dimension** had a subtler and more consequential effect. Organizing requirements along the transmitter/receiver axis made intuitive sense from a specification perspective - the ISO standard itself frames many requirements in terms of transmitter behavior and receiver behavior - and it was genuinely useful for thinking through which requirements belonged where. However, it also made a split-path implementation architecture look like the natural design strategy, simply because the requirements were literally organized along that split. The implication appeared to be: implement a TX module, implement an RX module, and map the TX requirements to the former and the RX requirements to the latter.

This turned out to be a red herring. The pitfalls of the split-path approach were not at all apparent from the requirements table alone. The table made the split architecture look clean and well-motivated. The problems - design drift between separately implemented FSMs, integration complexity, and unnecessary hardware duplication - only surfaced later, during integration. This is an important general lesson: the

structure of a requirements model can inadvertently bias architectural decisions in ways that are not immediately obvious, and the apparent naturalness of a design strategy that mirrors the requirements structure is not in itself a reliable signal that the strategy is sound.

6.2 Architectural Design Decisions

Before settling on the final architecture, several design alternatives were evaluated. The exploration drew on three sources: the existing in-house CAN Classic controller (Section 1.2), the open-source CTU CAN FD core [2], [3], and the ISO 11898-1 standard's own layered reference model [1]. The protocol requirements and engineering constraints documented in Section ?? bound the feasible design space; this section documents the key decisions made within it.

6.2.1 Monolithic vs. Layered Architecture

The most fundamental architectural decision was whether to extend the existing monolithic controller or adopt a layered decomposition.

The existing controller uses a flat architecture: a single FSM (`can_fsm`, 810 lines) directly drives the bus output, reads the bus input, manages bit counting, handles stuff bit insertion, coordinates CRC computation, and updates error counters - all in one clocked process. This approach minimizes inter-module latency and signal fanout, since every decision is made in a single combinational cone. For CAN Classic, where the frame has at most two format variants (base and extended) and a single bit rate, the complexity is manageable.

CAN FD, however, introduces four frame formats, dual bit rates, fixed bit stuffing with SBC encoding, three CRC polynomials, and a richer error model. Extending the monolithic FSM to cover these features would require nested conditionals on the format type in nearly every state, increasing the cyclomatic complexity well beyond what is practical to verify or maintain. The existing controller's `s_arbitration` state already contains 70 lines of interleaved TX/RX logic for two frame formats; scaling this to four formats with additional control fields (BRS, ESI, SBC) would roughly triple the state's complexity.

The alternative - and the approach adopted - is to follow the ISO 11898-1 reference model, which decomposes the data link layer into LLC, MAC, and PCS sub-layers with a separate FCE. Each sub-layer has a well-defined service interface (described in Section ??), and each can be implemented and verified independently. This decomposition introduces inter-module interfaces and pipeline latency, but it confines format-specific complexity to the module where it belongs: the MAC FSM handles frame field sequencing, the bit stuffer handles stuff bit insertion and SBC generation, and the CRC engine handles polynomial computation. No module needs to know about all three concerns simultaneously.

CTU CAN FD [2] takes a middle path: it separates protocol processing (in a `CAN Core` block) from buffer management and register access, but the protocol core itself remains a large monolithic unit. This is a reasonable trade-off for a general-purpose IP core that must support message filtering, multiple TX buffers, and DMA - features that require tight coupling between the protocol engine and the buffer subsystem. For the narrower scope of this project (protocol engine only, no message RAM or filtering), the full ISO layered decomposition is feasible and preferred because it directly maps each module to a testable subset of the standard's requirements.

6.2.2 Combined vs. Separated TX/RX FSMs

The initial design for the new MAC used **separate TX and RX FSMs**: `can_mac_fsm_tx` (~700 lines, 21 states) wrapped inside `can_mac_tx`, and `can_mac_fsm_rx` (~640 lines, 19 states) wrapped inside `can_mac_rx`. Each wrapper instantiated its own `can_mac_bs` and `can_mac_crc`. The two FSMs shared no state. The only coupling was a `transmitting_i` flag passed from TX to RX, and a dominant-wins OR-merge of their respective PCS outputs at the `can_mac` wrapper level.

The argument for the split was grounded in the verification plan structure. The AI-assisted extraction described in Section 3.3 had classified each requirement by side (TX, RX, both), layer, and frame format. Mapping that classification onto separate entities looked elegant: TX-side requirements would be verified by exercising `can_mac_tx` in isolation, RX-side requirements by exercising `can_mac_rx` in isolation, with no cross-path stimulus needed. The separation also appeared to offer independence - a bug in one path could not corrupt the other's state.

In practice the split created more problems than it solved. Frame structure - the field ordering, the bit stuffing rules, and the CRC polynomials - is identical regardless of which node is driving. Encoding it twice meant that a fix to FD CRC delimiter handling in the TX FSM had to be replicated in the RX FSM with no compiler enforcement that the copies remained in sync. The doubled submodule footprint similarly doubled investigation surface: every "is the bit stuffer handling fixed stuffing correctly?" question had to be answered for two independent instances.

The most concrete cost was a debugging episode in `can_mac_pcs_fce_tb`, where a node that had successfully transmitted a frame was reporting `c_disturbed` instead of `c_transmitted`. Tracing the bug required correlating TX FSM state, TX bit count, RX FSM state, RX bit count, BS state on both sides, CRC state on both sides, and the merged PCS output - all in the same waveform pane, across two parallel state vectors that re-derived the frame position independently. The session made clear that single-bit-time bugs need a single-bit-time view, and that two parallel FSMs fundamentally opposed that.

The deeper lesson concerns the relationship between verification plan structure and RTL structure. Verification plan dimensions - TX/RX side, layer, frame format - are inputs to **testbench architecture**, not to RTL architecture. They describe what must be tested and what stimulus configuration is needed to test it. RTL architecture should follow the structure of the protocol itself. The natural code unit of the MAC is the frame, not the side, because a frame has the same shape whether the local node is driving it or sampling it. Cutting the natural code unit at an artificial seam, then re-joining the halves with shared submodules and a dominant-wins merge, added coupling without removing complexity.

The **final design** uses a single unified `can_mac_fsm` entity: one synchronous process, one `t_fsm_state` enum (24 states), one shared `can_mac_bs`, one shared `can_mac_crc`, and one `is_transmitter` mode flag latched at Start-of-Frame. TX-only requirements are verified with `is_transmitter = true` stimulus, RX-only with `is_transmitter = false` - the verification plan dimensions map cleanly onto testbench configurations rather than onto separate RTL entities.

6.2.3 Per-Field vs. Per-Phase FSM Granularity

A second FSM design decision was the granularity of states. The existing controller uses per-phase states: `s_arbitration` covers all ID bits, SRR, IDE, and RTR; `s_control` covers reserved bits and DLC; `s_data` covers all data bytes. Within each state, a bit counter and conditional logic dispatch the correct action for each bit position.

This per-phase approach keeps the state count low (18 states in the existing controller) but pushes complexity into the bit-counting logic. The `s_arbitration` state must distinguish between base and extended formats using the bit counter, handle the SRR/IDE branching point at bit 12/13, and detect arbitration loss - all in a single state with a single counter.

The alternative is per-field states, where each protocol field (SOF, ID, SRR/RRS, IDE, FDF, RES, BRS, ESI, DLC, Data, SBC, CRC, ACK, EOF) has its own state. This increases the state count (19 TX states, 17 RX states) but eliminates most bit-counter conditionals: when the FSM is in `s_brs`, it knows exactly which bit it is processing without consulting a counter. Format-dependent transitions are handled by the state graph itself - for example, the FSM transitions from `s_ide` to `s_fdf_r1_r0` for all formats, but from `s_fdf_r1_r0` it may go to `s_dlc` (Classic) or `s_res_r0` then `s_brs` (FD). Each state contains only the logic relevant to its specific field, and named guard predicates (`v_in_arbitration_field`, `v_in_dynamic_stuff_field`, `v_in_fixed_stuff_field`) replace repeated bit-range comparisons.

The per-field approach was chosen because it aligns with the verification plan: each field in the frame structure corresponds to a set of requirements in the verification plan, and each FSM state can be traced directly to those requirements. It also simplifies formal verification, since PSL assertions can reference state names rather than counter ranges.

6.2.4 Interface Record Design

The company's `std_logic`-only port constraint does not preclude the use of VHDL record types on entity ports - records of `std_logic` and `std_logic_vector` fields satisfy the constraint. The design uses typed record interfaces (e.g., `t_can_mac_pcs_if_m2s`, `t_can_mac_fsm_bs_if_s2m`) to bundle related signals into a single port. Each record type has a corresponding reset constant (e.g., `c_can_mac_pcs_if_m2s_reset`), ensuring that every module can be reset to a known state without manually enumerating each field.

The alternative - flat port lists with individual `std_logic` signals - was used in the existing controller, where the FSM entity has 20 individual ports for its various sub-module interfaces. This approach becomes unwieldy as the number of inter-module signals grows: the new design has over 60 inter-module signals across its interfaces, and bundling them into records reduces the port list from dozens of lines to six record-typed ports per FSM entity.

The naming convention follows the data-flow direction: `m2s`/`s2m` (master-to-slave/slave-to-master) for control interfaces, and `s2d`/`d2s` (source-to-destination/destination-to-source) for Avalon-ST data-transfer interfaces. This convention is consistent with the company's existing interface naming and makes the direction of data flow explicit at every port.

6.2.5 Dual CRC Data Feeds

A non-obvious design decision concerns the CRC engine's data input. In CAN Classic, the CRC is computed over the raw bit stream excluding stuff bits. In CAN FD, the CRC is computed over the raw bit stream in most of the frame, but in the arbitration region the computation includes dynamic stuff bits - a difference from Classic that exists because the FD CRC must protect the stuff bit count as well as the data. This means that a single data feed to the CRC engine is insufficient: the Classic CRC needs the de-stuffed stream, while the FD CRC needs the stuffed stream during arbitration and the de-stuffed stream elsewhere.

The chosen solution exposes two data inputs on the CRC interface: `data_cc` (Classic CAN data, always de-stuffed) and `data_fd` (FD data, which includes dynamic stuff bits during the arbitration region). The FSM

drives both feeds, and the CRC wrapper routes `data_cc` to the CRC-15 engine and `data_fd` to the CRC-17 and CRC-21 engines. This avoids multiplexing logic inside the CRC module and keeps the CRC wrapper purely structural - it instantiates three `gen_crc` blocks and an output mux, with no protocol knowledge.

6.3 System Overview

A complete CAN node decomposes into a TX path and an RX path, coordinated by a shared Fault Confinement Entity (FCE) and Physical Medium Attachment (PMA) control, as shown in Figure 7. Each path spans three sub-layers - LLC (Section 6.5), MAC (Section 6.6), and PCS (Section 6.8) - with the LLC frame format defined in Section 6.4 and interface bundles defined in Section ??. A centralized types package (Section ??) defines all protocol constants and interface records. Within the MAC sub-layer, a unified `can_mac` wrapper (Section 6.6.2) instantiates `can_mac_fsm` and `can_fce`, so that the wrapper exposes only LLC and PCS interfaces plus the FCE's LLC and PCS interfaces.

6.4 LLC Frame Format

The `LLC Frame` format used by the existing CAN-bus implementation (`can_bus_controller`) is depicted in Figure 8 with the ID byte format depicted in Table 6. A revised `LLC Frame` supporting FD is depicted in Figure 9. The revised format adds a 3-bit `FMT` [1, Sec. 6.4.3] field to the DLC byte (`LLC Frame` byte 4), expands the data field to 64 bytes, and repurposes reserved bits in the last byte for BRS and ESI flags.

The `FMT` encodes the supported frame formats as '`000`' = CB, '`100`' = CE, '`010`' = FB, '`110`' = FE. Accordingly, for the frame content to be self-consistent the IDE bit must be set to `0` for `FMT` = CB/FB and `1` for `FMT` = CE/FE. The revised format is designed to be backward compatible. An implementation that only supports Classic frames can simply ignore the `FMT` bits and treat all frames as Classic, while an implementation that supports FD can use the `FMT` field and the additional control bits (BRS and ESI) to distinguish frame types without affecting the existing ID and data field structure.

Table 6: ID byte encoding for the LLC frame formats shown in Figure 8 and Figure 9.

Format	Byte ID0	Byte ID1	Byte ID2	Byte ID3
Basic	'`00000000`'	'`00000000`'	'`00000`' & '`ID(10-8)`'	'`ID(7-0)`'
Extended	'`000`' & '`ID(28:24)`'	'`ID(23-16)`'	'`ID(15-8)`'	'`ID(7-0)`'

6.5 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

6.5.1 `can_llc_tx`

`can_llc_tx` buffers an incoming LLC frame from the user, streams it byte-by-byte to the MAC serializer, and manages retransmission on bus disturbance up to the ISO-mandated limit [1, Sec. 6.4.5] and 6.5.

6.5.2 `can_llc_rx`

`can_llc_rx` receives a reconstructed LLC frame from the MAC layer, applies acceptance filtering, and delivers accepted frames to the user over the `llc_rx_if` Avalon-ST stream [1, Sec. 6.4.5].

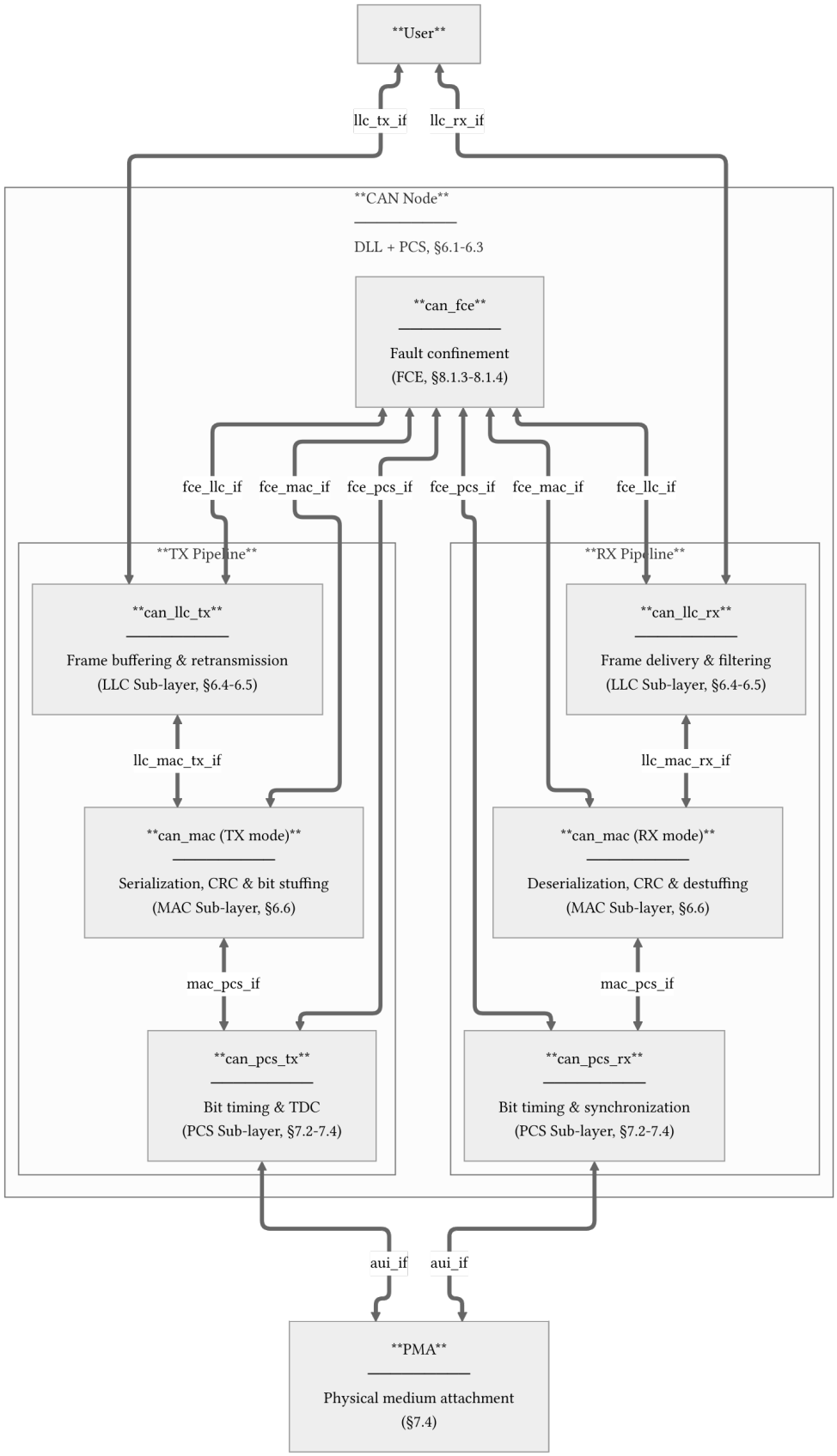

Figure 7: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for llc_tx_if (Table ??), llc_rx_if (Table ??), llc_mac_tx_if (Table ??), llc_mac_rx_if (Table ??), mac_pcs_if (Table ??), aui_if (Table ??), fce_llc_if (Table ??), fce_mac_if (Table ??), and fce_pcs_if (Table ??).

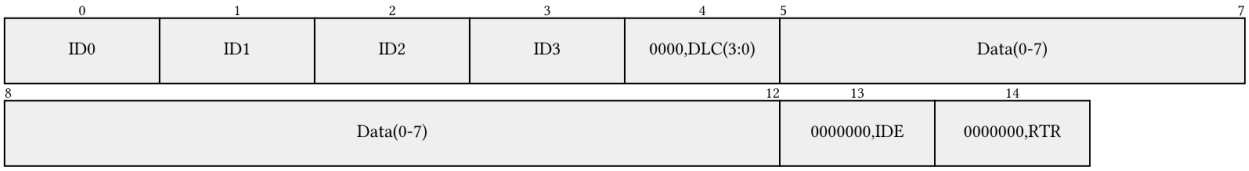

Figure 8: Current LLC frame format (15 bytes). ID byte encoding is defined in Table 6.

Figure 9: Revised LLC frame format with FD support, shown at maximum length (71 bytes). The FMT field selects frame type; BRS and ESI repurpose reserved bits in the final byte. ID byte encoding is defined in Table 6.

6.6 MAC Sub-layer

The MAC sub-layer is the core of the protocol logic, responsible for bit serialization, CRC generation, bit stuffing, and frame-level error detection. It coordinates closely with the FCE (Section 6.7) for error counter management and node-state transitions (Error Active/Passive/Bus Off), and with the PCS (Section 6.8) for sample-point-driven bit output.

The earlier CAN bus controller concentrated TX, RX, MAC, and FCE logic in a single monolithic FSM (`can_fsm`), with the sub-functions - serialization (`can_ast_to_serial`), bit stuffing (`can_stuff_bit_gen`), and CRC (`gen_crc`) - implemented in satellite modules driven directly by it. PCS timing logic was implemented in `can_node_clock`, which fed sample-point and transmit pulses into `can_fsm` - a strategy retained in the current design.

The current design restructures these modules around the ISO 11898-1 [1] layer boundaries: `can_ast_to_serial` becomes `can_mac_ser_tx` (Section 6.6.1.5), `can_stuff_bit_gen` becomes `can_mac_bs` (Section 6.6.1.6), `gen_crc` becomes `can_mac_crc` (Section 6.6.1.7), and `can_node_clock` becomes `can_pcs` (Section 6.8.1). A single unified `can_mac_fsm` entity handles both TX and RX roles, sharing one `can_mac_bs` and one `can_mac_crc` instance across both paths. The FSM stores received bits directly in an internal frame array and streams the completed frame to the LLC during intermission, eliminating the need for a separate deserializer module. Fault confinement, previously embedded in `can_fsm`, is separated into its own entity (`can_fce`) and wired through the `can_mac` wrapper (Section 6.6.2).

6.6.1 `can_mac_fsm`

The `can_mac` sub-layer is built around a single unified FSM entity (`can_mac_fsm`, ~1100 lines) that handles both frame transmission and frame reception. The architecture is shown in Figure 10.

6.6.1.1 FSM structure and mode flag `can_mac_fsm` contains one synchronous process and one `t_fsm_state` enum covering 24 states. An `is_transmitter` boolean signal is latched to `true` when the FSM drives the SOF dominant bit at the start of a new frame transmission and cleared at arbitration loss or on any transition back to `s_bus_idle`. Once latched, `is_transmitter` remains stable for the entire frame, partitioning per-state logic into a TX branch and an RX branch without duplicating the state graph itself. States that genuinely diverge between roles - `s_ack`, `s_ack_delimiter`, `s_eof` - carry an explicit `if is_transmitter` branch. States that are behaviorally identical for both roles (which is most of the frame-field sequence) execute a single shared code path and read `rx_data` from the PCS, relying on the bus loopback guarantee that the winning transmitter's own echo matches its drive in the nominal phase.

The per-field state granularity introduced in Section 6.2.3 is preserved in the unified FSM. Each protocol field (SOF, ID, RTR/SRR/RRS, IDE, FDF, RES, BRS, ESI, DLC, Data, SBC, CRC, ACK, ACK Delimiter, EOF) has its own dedicated state, making field boundaries explicit in the state encoding without bit-counter conditionals.

6.6.1.2 TX mode: frame transmission When `is_transmitter = true`, the FSM drives bits through `pcs_o.tx_data` at the bit-boundary strobe (`drive_bit`, generated internally by registering `pcs_i.sample_point` twice). On each sample-point strobe the FSM executes the following four-step sequence per frame-field state:

1. **Monitor the bus.** The sampled bus polarity (`pcs_i.rx_data`) is compared against the previously transmitted bit (held in a 32-bit polarity history shift register, `transmitted_bits_shift_reg`) to

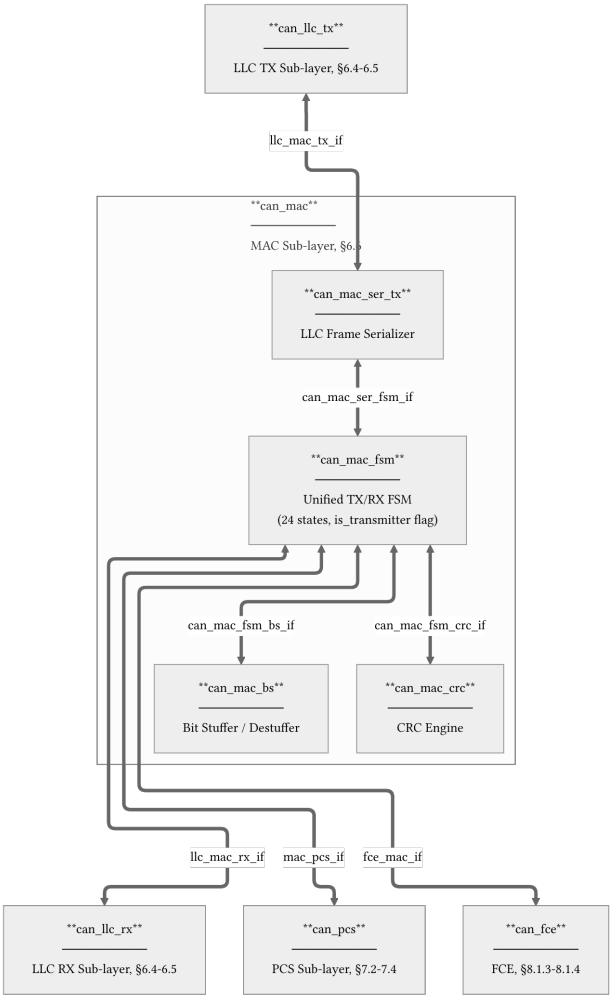

Figure 10: can_mac architecture showing the unified can_mac_fsm and its internal submodules. The FSM instantiates one can_mac_ser_tx serializer, one can_mac_bs bit stuffer, and one can_mac_crc CRC engine, shared across TX and RX modes. Internal interface definitions are provided for can_mac_ser_fsm_if (Table ??), can_mac_fsm_bs_if (Table ??), and can_mac_fsm_crc_if (Table ??).

detect bit errors, ACK, and arbitration loss. In the CAN-FD data phase, any pending secondary sample point (SSP) observation from the previous bit period is evaluated against this history before the current bit is checked [1, Sec. 7.3.4]. A detected error triggers a transition to the error-frame sequence with the appropriate flag type based on FCE fault confinement status [1, Secs. 8.1.3–8.1.4]. Arbitration loss causes `is_transmitter` to flip to `false` in-place, and the `s_arbitration` state then continues as an RX observer for the remaining bits.

2. **Determine the next bit.** If the bit stuffer has a pending stuff bit (`bs_i.valid`), that takes priority. Otherwise, the polarity is determined by the current state: form bits (SOF, IDE, FDF, reserved, delimiters, EOF) have fixed polarities, while ID, DLC, data, SBC, and CRC bits are sourced from the serializer, metadata, stuff bit count, or CRC register respectively.

3. **Feed the CRC engine and bit stuffer.** In the nominal (arbitration and control) phase, `pcs_i.rx_data` feeds both the bit stuffer and the CRC engine - the bus echo of the winning transmitter is identical to its drive, so this feed is correct regardless of role. In the FD data phase, where TDC delay may cause `rx_data` to lag the drive by several bit periods, the TX path instead feeds the bit stuffer and CRC engine from `transmitted_bits_shift_reg(0)` (the bit just driven), avoiding latency-induced stuff bit misplacement. The FSM asserts `fixed_bit_stuffing_en` when entering the SBC/CRC field region of FD frames to switch the bit stuffer from dynamic to fixed mode.

4. **Present the bit at the PCS interface.** The resolved polarity is written to `pcs_o.tx_data` and becomes visible at `tx_o` when the bit-boundary latch fires. The `use_data_rate` signal is asserted during the CAN-FD data phase to switch the PCS to the faster bit timing, and `start_tdc` is pulsed at the FDF bit to initiate transmitter delay compensation [1, Sec. 7.3.4].

6.6.1.3 RX mode: frame reception

When `is_transmitter = false`, the FSM observes `pcs_i.rx_data` at each sample-point strobe and stores received bits directly into an internal `llc_frame` byte array. No separate deserializer is needed. The bit stuffer is driven from `pcs_i.rx_data` to perform destuffing, and the CRC engine accumulates the received bit stream in parallel. The FSM validates the SBC field (FD frames), compares the received CRC against the locally accumulated result, and checks form bits (reserved bits, CRC delimiter, ACK delimiter, EOF) for required polarities. A mismatch in any of these fields triggers a transition to the error-frame sequence. During the ACK slot the FSM drives `pcs_o.tx_data = c_dominant` (1 bit for CC frames, 2 bits for FD frames) and releases the bus after the ACK delimiter [1, Sec. 8.1.4.2.b].

After the EOF field the FSM transitions through `s_intermission`. A dedicated streaming section within the same process transfers the completed `llc_frame` array byte-by-byte to the LLC RX sink over the Avalon-ST interface, and signals successful reception to the FCE. This design eliminates the need for a separate deserializer entity on the RX path - the frame buffer and the LLC stream interface are managed entirely within `can_mac_fsm`.

6.6.1.4 Error-frame states

The unified FSM uses four explicit error-frame states rather than a single compressed state, following the structure of the reference `can_fsm` controller. The split makes the four ISO-defined error-frame phases directly visible as state names in GTKWave and eliminates an ambiguity that existed in a two-state predecessor: a dominant during the recessive delimiter (a form error per ISO 6.6.21.3.2) and a dominant from another node's late error flag (tolerated per ISO 8.1.4.2.f) previously shared one code path. The four states are:

Table 7: Four explicit error-frame states of `can_mac_fsm`.

State	ISO reference	Role
`s_error_flag`	§6.6.13, §6.6.5.2	Drive 6 dominant bits (active) or 6 recessive bits (passive).
`s_error_flag_check`	§8.1.4.2.b	Evaluate the first bit after the flag; detect co-signalled errors.
`s_error_dominant_delim`	§8.1.4.2.f	Count tolerated dominant bits from other nodes during the delimiter.
`s_error_delimiter`	§6.6.5.3	Count 8 recessive delimiter bits; a dominant here starts a fresh error frame.

The complete FSM is shown in Figure 11.

6.6.1.5 `can_mac_ser_tx` `can_mac_ser_tx` converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM (Figure 12) manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization. The serializer extracts LLC metadata (IDE, FDF, DLC, FTYP, BRS, ESI) from the two config bytes and registers it in `t_llc_metadata`, which remains stable for the entire frame. The serializer also handles ID field padding - skipping unused bits in the 32-bit ID field for 11-bit base identifiers - and forwards `transfer_status` from the FSM back to the LLC to terminate serialization on error or completion.

6.6.1.6 `can_mac_bs` `can_mac_bs` implements both dynamic and fixed bit stuffing for CAN Classic and CAN-FD frames [1, Sec. 10.6]. The single entity is instantiated once inside `can_mac_fsm` and is used for both TX stuffing and RX destuffing, controlled by the FSM's `is_transmitter` mode and the frame-field state.

In **dynamic mode** (`fixed_bit_stuffing_en` = '0'), the stuffer monitors the polarity stream and inserts an inverse-polarity stuff bit after every five consecutive identical bits (ISO 6.6.13.2). In **fixed mode** (`fixed_bit_stuffing_en` = '1'), used for the FD CRC region, one fixed stuff bit (FSB) is inserted on the rising edge of `fixed_bit_stuffing_en`, then one every four real bits (ISO 6.6.13.3.1). Any dynamic stuff bit pending at the mode boundary is suppressed. The stuffer maintains a Gray-coded stuff bit count with parity for the SBC field via `f_to_gray()` and `f_calc_parity()`. The data path diagram is shown in Figure 13.

6.6.1.7 `can_mac_crc` `can_mac_crc` provides CRC generation and checking for both CAN Classic and CAN-FD frames. CAN Classic frames use CRC-15, while CAN-FD frames use CRC-17 (data payloads up to 16 bytes) or CRC-21 (data payloads above 16 bytes) [1, Sec. 10.4.2.6]. The single entity is instantiated once inside `can_mac_fsm`, serving both TX (generation) and RX (checking). The FSM selects the appropriate polynomial via the `crc_poly_select` field (Table **??**). The data path diagram is shown in Figure 14.

Three parallel `gen_crc` instances run continuously on separate data feeds: `data_cc` drives CRC-15 via `valid_cc`, while `data_fd` drives both CRC-17 and CRC-21 via `valid_fd`. This dual-feed architecture is necessary because CC and FD frames compute CRC over different bit streams (CC excludes stuff bits; FD includes them in the arbitration region), and the RX path does not know which CRC engine to use until

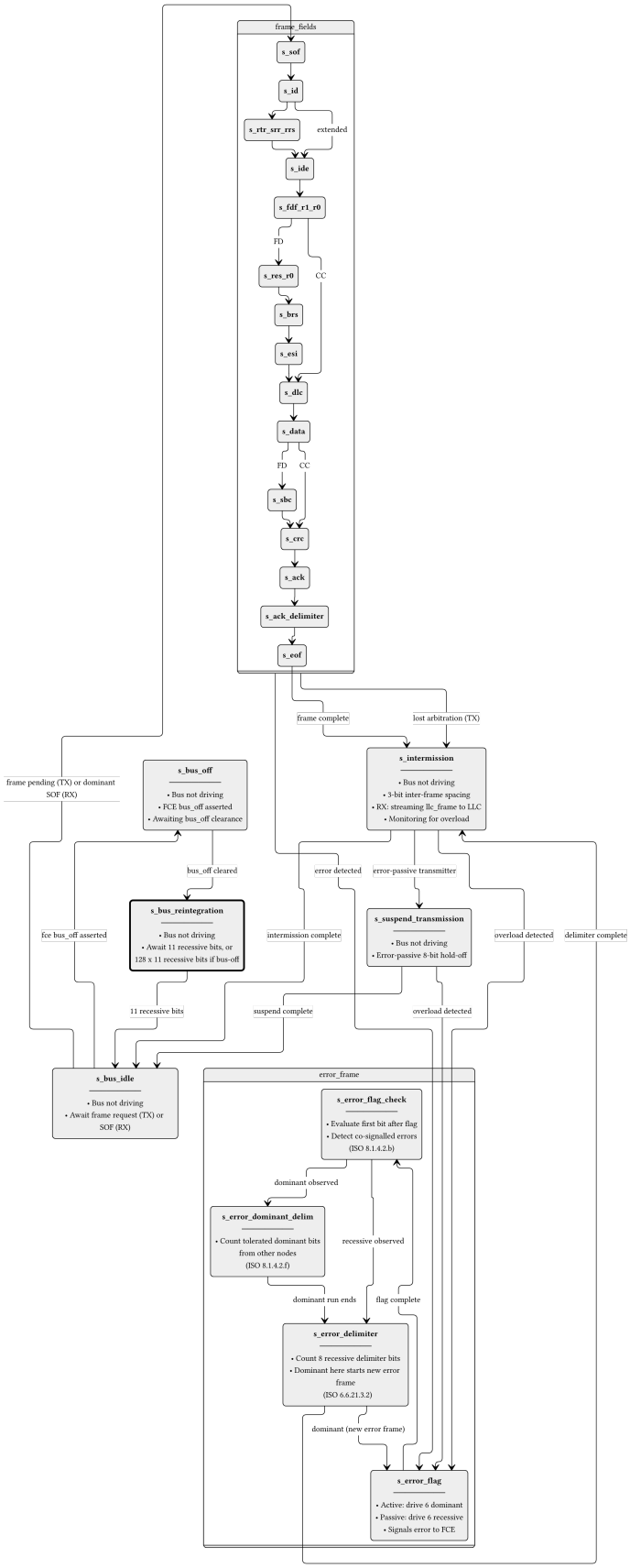

Figure 11: can_mac_fsm state diagram with per-field granularity (24 states). TX mode is active when is_transmitter = true (latched at SOF drive); RX mode when is_transmitter = false. Frame-field states (s_sof through s_eof) each handle one protocol field with bit_count tracking position within the field. After a successful frame, arbitration loss, or error recovery, the FSM passes through s_intermission before returning to s_bus_idle. Detected errors branch to the four-state error-frame sequence (s_error_flag through s_error_delimiter). The s_bus_off state is entered when the FCE asserts bus_off and exits when bus_off clears. The dashed box groups the frame-field states for clarity.

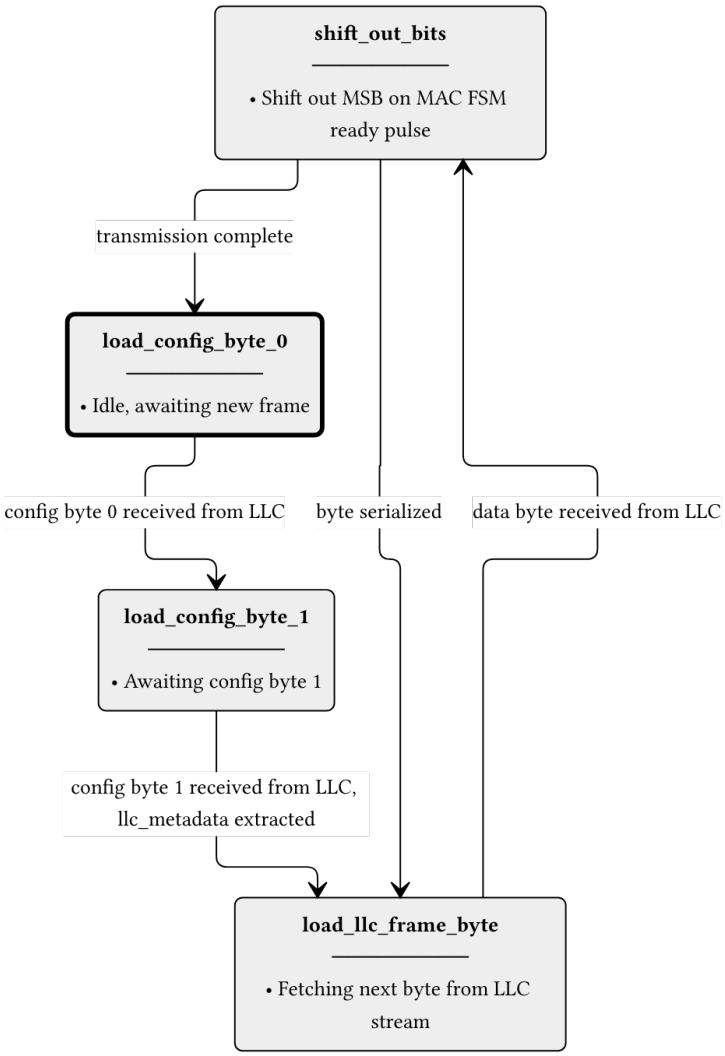

Figure 12: can_mac_ser_tx FSM. The serializer accepts LLC frame bytes over a ready/valid handshake and shifts them out MSB-first to the MAC FSM one bit per ready pulse. LLC metadata is extracted from the two config bytes and registered in t_llc_metadata for use by the FSM throughout the frame.

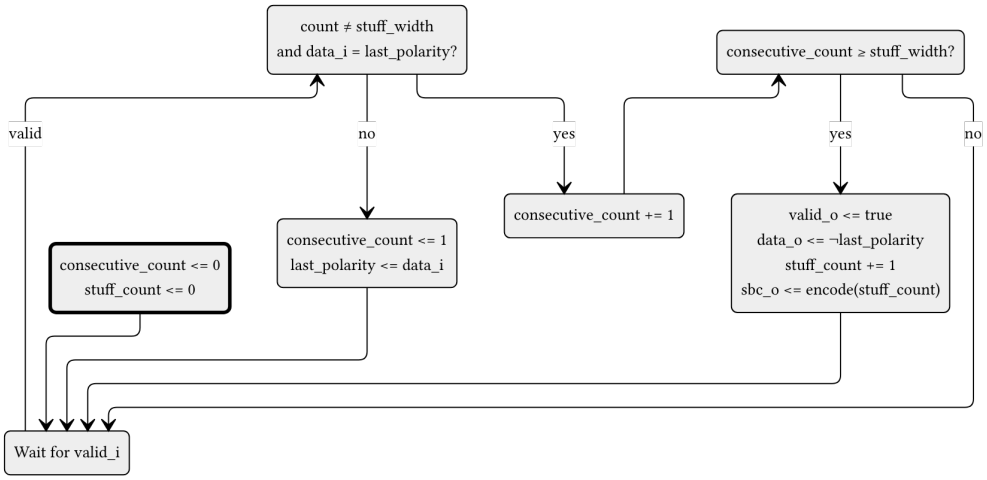

Figure 13: can_mac_bs data path diagram. When valid and data = last_polarity, consecutive_count increments. Otherwise it restarts at 1 with last_polarity updated to data. When consecutive_count reaches stuff_width (5 in dynamic mode, 4 in fixed mode), valid and inverted data are driven and stuff_count increments. The to_gray() + calc_parity() block encodes stuff_count into the stuff_bit_count output. The external rst signal resets consecutive_count and stuff_count to zero. The fixed_bit_stuffing_en signal selects between dynamic and fixed modes. All outputs are registered.

after the frame type has been determined. The output multiplexer selects the active engine's result based on `crc_poly_select` and left-aligns it to the common 21-bit output width.

6.6.2 `can_mac` Unified Wrapper

`can_mac` is a structural wrapper that instantiates `can_mac_fsm` and `can_fce` (Section 6.7). Because both TX and RX roles are handled by the single FSM entity, the wrapper requires no dominant-wins merge of separate PCS outputs and no multiplexing of separate FCE error records. The FSM drives one PCS interface and one FCE interface directly. The wrapper exposes LLC TX and RX interfaces (connected to `can_mac_ser_tx` inside the FSM and to the FSM's internal LLC RX stream respectively), one `mac_pcs_if` interface, and the FCE's LLC and PCS interfaces.

6.7 FCE Sub-layer

The Fault Confinement Entity (`can_fce`) implements the error state machine and counter management specified in [1, Secs. 8.1.3–8.1.4]. It maintains two error counters - TEC (Transmitter Error Counter, 0-511) and REC (Receiver Error Counter, 0-255) - and transitions between three states: `s_error_active` (normal operation), `s_error_passive` (TEC or REC > 127), and `s_bus_off` (TEC > 255).

Counter updates follow the ISO 8.1.4.2 rules: TEC increments by 8 on TX errors (unless `counters_unchanged` is asserted), decrements by 1 on successful TX. REC increments by 1 on RX errors during non-error-flag phases, by 8 on primary errors or error-flag-phase errors, and decrements by 1 or clamps to 127 on successful RX. The FCE state machine (Figure 15) transitions between error-active, error-passive, and bus-off based on counter thresholds. Bus-off recovery requires counting 128 idle conditions (11 consecutive recessive bits each) from the PCS via the `idle_condition` signal, or a `normal_mode_request` from the LLC.

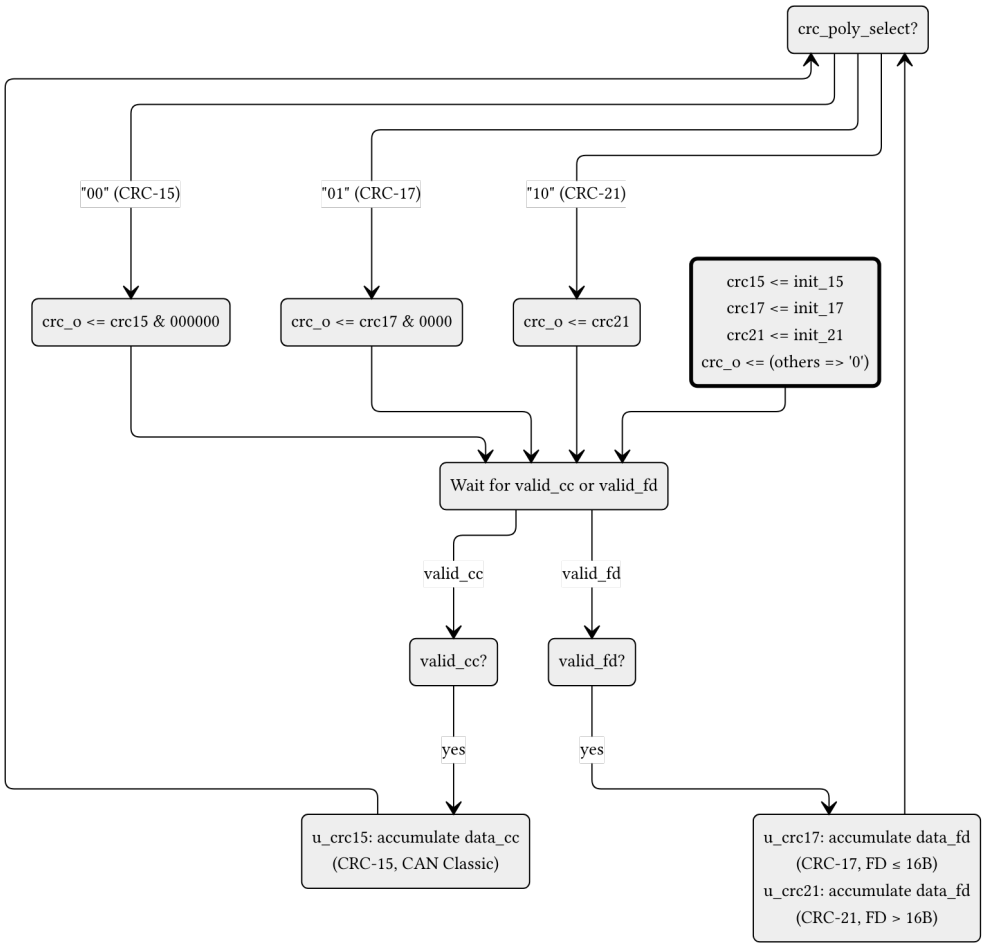

Figure 14: can_mac_crc data path diagram. Three parallel gen_crc instances accumulate continuously: data_cc drives CRC-15 via valid_cc, while data_fd drives both CRC-17 and CRC-21 via valid_fd. The output register selects the active engine result based on crc_poly_select and left-aligns it to 21 bits. The external rst signal reinitializes all three engines and the output register.

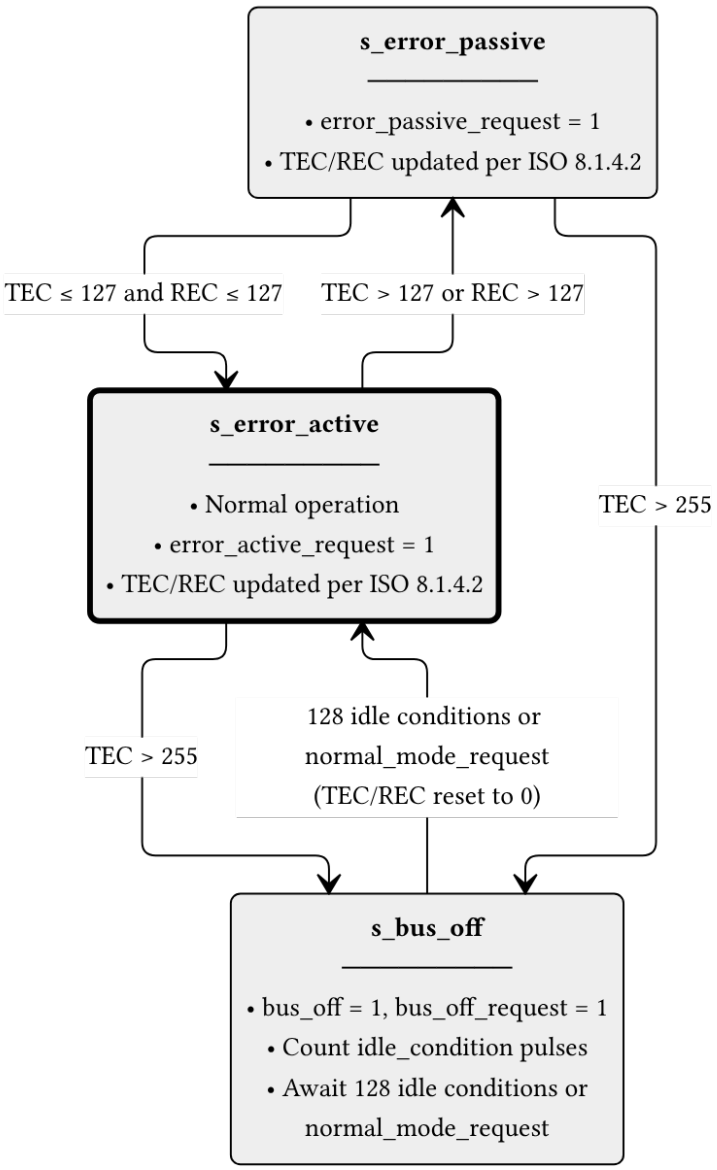

Figure 15: can_fce FSM (ISO 8.1.4.1, Figure 43). The s_error_active state is the normal operating mode. When either TEC or REC exceeds 127, the FCE transitions to s_error_passive and asserts error_passive_request to both MAC paths. If TEC exceeds 255, the node enters s_bus_off: the FCE issues bus_off_request to the PCS and bus_off to the LLC. Recovery from bus-off requires 128 idle conditions (each 11 consecutive recessive bits) counted from the PCS, or a normal_mode_request from the LLC, after which the counters are reset and the FCE returns to s_error_active.

6.8 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

6.8.1 `can_pcs_tx`

`can_pcs_tx` handles bit timing and Transmitter Delay Compensation (TDC) for the TX path [1, Secs. 7.2–7.3]. It generates the sample point (SP) strobe used by the MAC FSM to advance frame state, and - when TDC is configured - a secondary sample point (SSP) strobe for data-phase bit monitoring. The FSM (Figure 16) has four states reflecting the two bit rates and the TDC measurement window. The `measuring_delay` state determines the Transmitter Delay Compensation Value (TDCV, [1, Sec. 7.3.4]) by observing the TX-to-RX propagation delay, which together with the configured TDCO offset defines the SSP position for subsequent data-phase bits.

6.8.2 `can_pcs_rx`

`can_pcs_rx` implements bit timing and synchronization for the RX path. It performs hard and soft synchronization on the incoming bus signal and generates the sample point strobe used by the MAC RX layer to latch each received bit [1, Secs. 7.2–7.3].

---

7 Implementation

7.1 Type Safety and Packages

The implementation uses custom record types (defined in `can_types_pkg.vhd`) to ensure clean interfaces between modules.

7.2 Bit Timing and TDC

Detailed description of how `tx_pcs` measures propagation delay and calculates the SSP.

7.3 CRC and Bit Stuffing

Implementation details of the flexible CRC generator and the hybrid bit stuffer.

8 Verification and Results

Note: This section is currently being populated as the verification plan is executed.

8.1 Testbench Results Summary

Table 8: Testbench execution status and functional coverage summary.

Testbench	Status	Coverage
`tx_pcs_tb`	Pass	95%
`tx_can_tb`	Pass	88%
`tx_can_protocol_tb`	Pass	92%

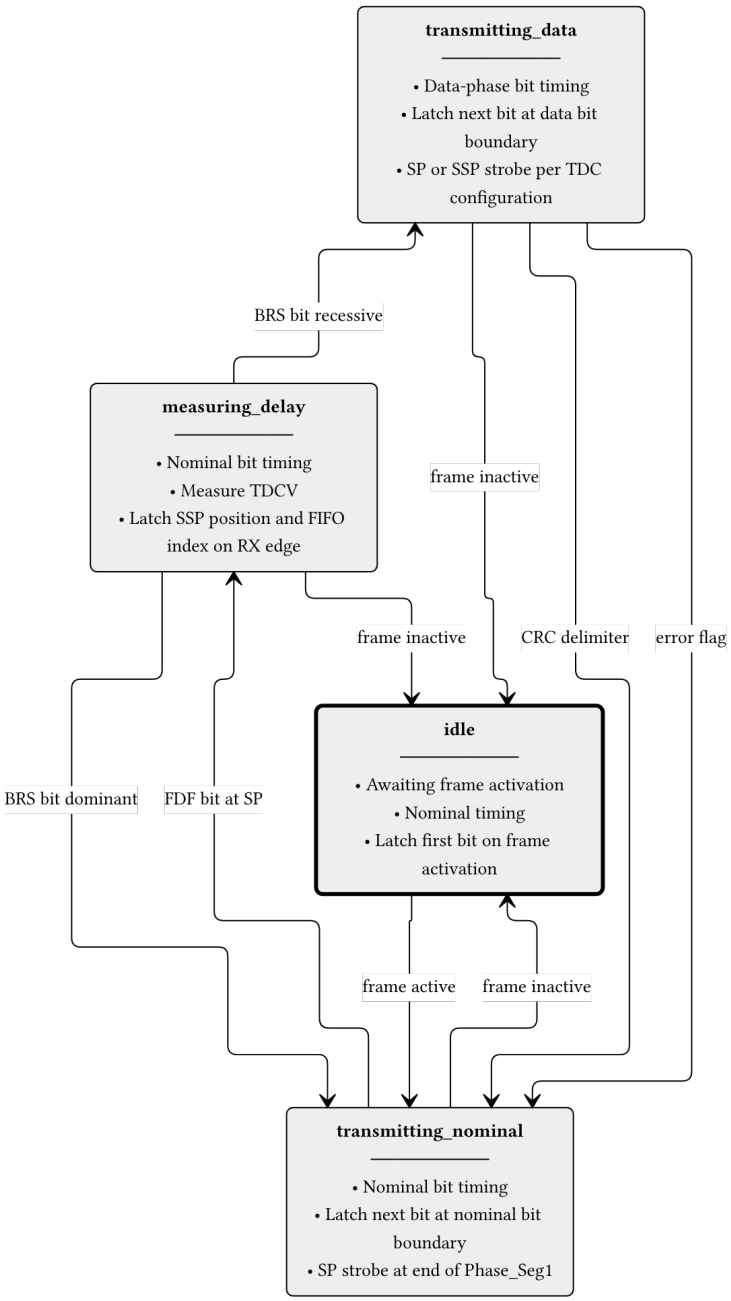

Figure 16: can_pcs_tx FSM. The measuring_delay state is entered on the FDF sample point to measure TDCV (Transmitter Delay Compensation Value, [[1], sec. 7.3.4]). The BRS bit boundary determines whether data-phase timing is used. All non-idle states return to idle when the frame becomes inactive.

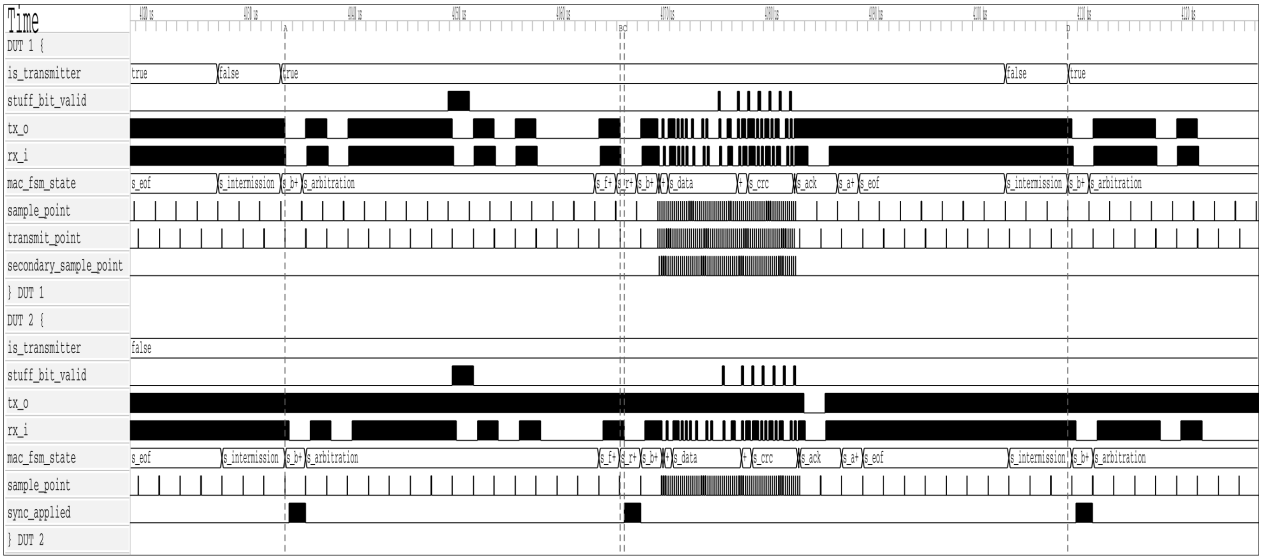

Figure 17: REQ-009.

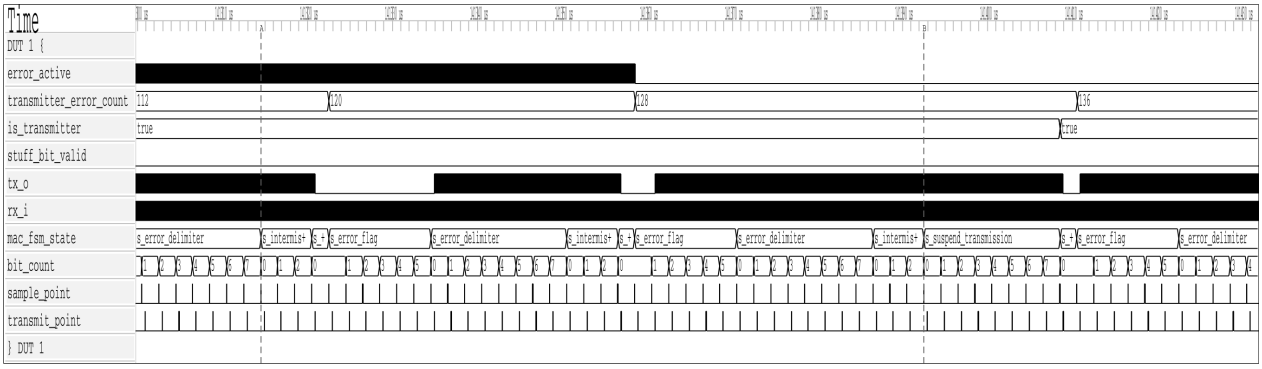

Figure 18: REQ-010.

9 Discussion

Comparison of the implemented architecture against theoretical models. Performance analysis in high-load scenarios.

9.1 Future Work

1. Make the the CRC and BS modules to save area on the FPGA.
2. CAN XL implementation…

10 Conclusion

Summary of work completed and how objectives were met.

11 References

A Verification Plan

Both tables are regenerated automatically from `verification_plan/verification_plan.toml` on each PDF build. The ID field is the join key between them. See Section 5.6 for the meaning of each field.

A.1 Extracted Requirements

The requirements table contains four fields: the identifier, the ISO 11898-1 source clause, the priority assignment, and the paraphrase. Placing priority here allows each assignment to be evaluated directly against the requirement text. The table corresponds to the requirements extraction and prioritisation phases described in Section 3.2 and Section 5.5.

Table 9: ISO 11898-1 extracted requirements.

ID	ISO ref	Priority	Requirement
REQ-001	6.4.5.2	P1	**LLC notification** LLC shall notify the user on every successful DF/RF TX or RX.
REQ-002	6.4.5.5.2, 6.4.5.5.3	P2	**LLC transmission request and abort timing** 1) A TX request shall be processed no later than the second SOF after the request when no EFs are present. 2) Frames not yet passed to the MAC are aborted immediately. 3) An abort request shall be processed before the second SOF (no EFs) or third SOF (with EFs).
REQ-003	6.5.2, 6.5.3	P1	**Retransmission** 1) Frames losing arbitration shall be retransmitted up to the configured limit unless aborted. 2) Non-XL nodes need not implement a retransmission counter but shall support unlimited retransmissions. 3) If implemented, the retransmission counter shall reset to zero on successful TX or on a new LLC TX request.
REQ-004	6.5.4	P3	**Frame acceptance filtering** 1) Receivers should use acceptance filtering to decide frame relevance. 2) Filtering should be a single self-contained operation with no state carry-over from previous frames.
REQ-005	6.5.6	P1	**LLC transfer status reporting** LLC shall report five transfer status values: Ongoing, Transmitted, Lost_Arbitration, Disturbed, and Aborted, each reflecting the corresponding MAC sub-layer event.
REQ-006	6.6.4.4	P1	**CRC rules** 1) CRC_15 for CC frames. CRC_17 for FD frames with payload ≤16 B. CRC_21 for FD frames with payload >16 B. 2) Init vector: all-zero for CRC_15. MSB-one for CRC_17 and CRC_21.

ID	ISO ref	Priority	Requirement
REQ-007	6.6.5.2, 6.6.5.3, 6.6.6.2, 6.6.6.3	P1	**Error and overload flags** 1) Active error and overload flags: 6 consecutive dominant bits each. 2) Passive error flag: 6 consecutive recessive bits (may be overwritten by dominant bits from other nodes). 3) Error and overload delimiters: 8 recessive bits each. 4) After the flag, all nodes monitor the bus and simultaneously begin their 7-bit delimiter on detecting the first recessive bit.
REQ-008	6.6.7.1, 6.6.7.2, 6.6.7.3, 6.6.7.4	P1	**Inter-frame space** 1) Intermission shall be exactly 3 recessive bits. 2) No DF or RF may be started during intermission. 3) EFs and OFs shall not be preceded by inter-frame space. Consecutive OFs shall not be separated by inter-frame space. 4) A pending frame shall be started at the first bit following intermission. 5) Bus idle is recognised at the third recessive intermission bit (error-active nodes/receivers), the eighth recessive suspend bit (error-passive transmitters), or on leaving bus-integration state. 6) An error-passive transmitter shall suspend for 8 bit times after intermission. If another node transmits during the suspend window, the node shall become a receiver.
REQ-009	6.6.7.5	P1	**Bus re-integration** 1) Nodes shall start bus re-integration on protocol startup or bus-off recovery. 2) Re-integration completes after 11 consecutive recessive bits at the sample point. 3) For FD-capable nodes, any synchronisation-causing edge shall reset the 11-bit recessive-bit counter.
REQ-010	6.6.7.3, 6.6.8	P1	**Frame initiation** 1) SOF shall be a single dominant bit. 2) A node shall send SOF only when the bus is idle. 3) A dominant third intermission bit causes a node with a pending frame (if error-active or previous receiver) to skip SOF and begin arbitration immediately. 4) A dominant bit detected during bus idle shall be interpreted as SOF.
REQ-011	6.6.10.1, 6.6.10.4	P2	**Remote frame** 1) RTR bit shall be dominant for DFs and recessive for RFs. 2) RFs have no data field. DLC indicates the requested data length.

ID	ISO ref	Priority	Requirement
REQ-012	6.6.10.2, 6.6.11.2	P2	**SRR and RRS reserved bits** 1) SRR (CE, FE): transmitted recessive. Receivers accept either polarity without form error. 2) RRS (FB, FE): transmitted dominant. Receivers accept either polarity without form error.
REQ-013	6.6.10.5, 6.6.11.5	P1	**CRC bit stream coverage** 1) CC: SOF, arbitration, control, and data fields (dynamic stuff bits excluded). 2) FD: SOF, arbitration, control, data, dynamic stuff bits, and stuff count field (fixed stuff bits excluded).
REQ-014	6.6.10.6, 6.6.11.6, 6.6.14	P1	**Frame acknowledgement** 1) Receivers shall ACK consistent frames and flag inconsistent frames with an EF. 2) Transmitters shall flag unacknowledged frames with an EF. 3) FD frames tolerate up to a 2-bit dominant ACK overlap to compensate for receiver phase differences. 4) The ACK delimiter shall be one recessive bit. The ACK slot is surrounded by recessive bits on both sides. 5) ACK shall not depend on the frame identifier.
REQ-015	6.6.10.3, 6.6.11.2, 6.6.11.3	P1	**FDF bit and CC/FD format differentiation** 1) FDF: dominant in CC frames, recessive in FD frames. 2) The first two CC control field bits (IDE in CBFF, r1/r0 in CEFF) shall be transmitted dominant. 3) A recessive r0 (CBFF) or r1 (CEFF) shall generate a form error. 4) In FBFF the IDE bit is the first control field bit. In FEFF it is in the arbitration field after SRR. 5) The res bit (FB, FE) shall be dominant. A recessive res bit is a form error.
REQ-016	6.6.11.3	P2	**ESI bit transmission** 1) Recessive when the LLC ESI flag is set. 2) Dominant when LLC ESI flag is clear and the node is error-active. 3) Recessive when LLC ESI flag is clear and the node is error-passive.
REQ-017	6.6.11.5	P1	**Stuff count (SBC) field** The SBC field shall be Gray-coded with a parity bit and placed before the FCRC, encoding the count of dynamic stuff bits in the frame.

ID	ISO ref	Priority	Requirement
REQ-018	6.6.10.5, 6.6.11.5	P2	**CRC delimiter** 1) CC frames: exactly one recessive CRC delimiter bit. 2) FD frames: transmitter sends one recessive bit. A second recessive bit is tolerated before the ACK slot.
REQ-019	6.6.13.1, 6.6.13.2, 6.6.13.3.1	P1	**Bit stuffing** 1) After 5 consecutive identical bits, a complementary dynamic stuff bit shall be inserted. The receiver shall discard it. 2) CC stuffing covers SOF through the FCRC sequence. FD dynamic stuffing covers SOF through the data field only. 3) The ACK field and EOF shall not be dynamically stuffed. 4) FD frames (FB, FE): a fixed stuff bit precedes the SBC field and follows every fourth CRC bit, each the inverse of the preceding bit. If the preceding field ends with 5 consecutive identical bits, only the FSB is emitted. A receiver detecting a fixed stuff bit with the same polarity as the preceding bit shall flag a form error.
REQ-020	6.6.10.7, 6.6.11.7, 6.6.15.2	P1	**End of frame (EOF)** 1) EOF shall consist of exactly 7 recessive bits. 2) A receiver shall validate a frame after the sixth EOF bit. 3) A transmitter requires all seven EOF bits to be error-free.
REQ-021	6.6.10.2, 6.6.11.2, 6.6.17.4, 6.6.17.5	P1	**Bus arbitration** 1) Recessive-sent, dominant-monitored: lose arbitration and become a receiver. 2) Dominant-sent, recessive-monitored: bit error. 3) Arbitration spans from the first ID bit to: IDE (CB), RTR (CE), or the bit after FDF (FB/FE). 4) When CB and FB frames with the same base ID collide, the CB frame wins via its dominant r0/FDF bit. 5) When a CBFF/FBFF and CEFF/FEFF frame with the same 11-bit base ID collide, the CBFF/FBFF frame wins via the recessive SRR/IDE bits. 6) (CB, CE only) When a DF and RF with the same ID and format collide, the DF wins via its dominant RTR bit.

ID	ISO ref	Priority	Requirement
REQ-022	6.6.21.2	P1	**MAC error detection** 1) Bit error: any sent/monitored mismatch. Exceptions: recessive non-stuff bits during arbitration; any recessive bit during the ACK slot; dominant bits monitored while sending a passive error flag. 2) Stuff error: detected at the sixth consecutive identical bit in any dynamically-stuffed field. 3) Form error: any unexpected fixed-form bit value (all formats) or a fixed stuff bit not the inverse of the preceding bit (FB, FE). Exception: a dominant bit at the last EOF bit or last delimiter bit is not a form error. 4) CRC error: FCRC mismatch. FD frames additionally require SBC match and correct SBC parity. 5) ACK error: transmitter detects no dominant bit in the ACK slot.
REQ-023	6.6.21.3.1	P1	**Error flag initiation and FCRC error behaviour** 1) On any detected error, start an EF at the immediately following bit and inform the LLC. 2) Data-phase errors require a switch back to nominal bit rate before the EF starts. 3) On an FCRC error, the receiver shall withhold ACK. 4) On an FCRC error, the receiver sends an EF at the first EOF bit (CC) or 3 bit-times after the CRC delimiter (FD).
REQ-024	6.6.21.3.2	P2	**Overload frame** 1) LLC-requested OF: starts at the first intermission bit. Optional, at most 2 per inter-frame space. 2) Reactive OF: triggered by a dominant bit at either of the first two intermission bits, the last EOF bit (receivers only), or the last error/overload delimiter bit. Starts one bit after the triggering bit.
REQ-025	7.3.2, 7.3.3	P1	**PCS bit timing configuration** 1) Separate nominal and FD data bit rate configurations (XL excluded), each based on a programmable integer prescaler. 2) Sample point shall be at the end of Phase_Seg1. 3) Sync_Seg shall be exactly 1 tq. 4) IPT ≤ 2 tq. SJW ≤ min(Phase_Seg1, Phase_Seg2) in both phases. 5) In FD data bit time, Phase_Seg2 ≥ SJW and ≥ maximum IPT.
REQ-026	7.3.2	P2	**Propagation segment sizing** prop_seg ≥ t_node(A) + t_node(B) + 2 × t_busline, where each node delay includes transceiver round-trip delay.

ID	ISO ref	Priority	Requirement
REQ-027	7.3.4, 6.6.21.3.1	P1	**Transmitter delay compensation** 1) FD-capable nodes shall implement TDC for the FD data phase when BRS is recessive, with programmable enable and prescaler constrained to 1 or 2. 2) SSP position = measured transmitter delay + configured offset. 3) When TDC is active, bit errors at the regular SP are ignored. Errors at the SSP are reacted upon at the following nominal SP.
REQ-028	7.3.5.1, 7.3.5.2, 7.3.5.3, 7.3.5.4	P1	**PCS synchronisation** 1) Only one synchronisation per bit time. Re-enabled after the next recessive sample point. 2) Edges qualify only if the previous sample point was recessive and no positive phase error exists while sending dominant. 3) Hard synchronisation occurs on IFS edges (except the first bit of intermission), during bus-integration, and at the FDF-to-res transition. Correction is unconstrained by SJW and restarts the bit time with Sync_Seg completed. 4) All other qualifying edges cause resynchronisation, except during the FD data phase. Positive phase error: Phase_Seg1 lengthened by SJW. Negative phase error: Phase_Seg2 shortened by SJW. When error ≤ SJW the effect equals hard synchronisation.
REQ-029	7.3.6	P2	**Oscillator frequency tolerance** 1) df < SJW / (20 × tbit) and df < min(Phase_Seg1, Phase_Seg2) / (2 × (13 × tbit − Phase_Seg2)). 2) SJW shall not exceed min(Phase_Seg1, Phase_Seg2).
REQ-030	6.6.7.5, 7.4.2.2, 7.4.2.3, 8.1.4.4	P1	**PCS bus-off isolation and signalling** 1) During bus-off, the PCS shall not drive dominant bits. 2) The PCS counts consecutive recessive bits at the sample point and strobes idle_condition after 11, resetting on any dominant. 3) The PCS shall disconnect the node from the bus on a bus_off_request. 4) The PCS shall reconnect on a bus_off_release_request.
REQ-031	8.1.3.2, Table 14, Table 15	P2	**FCE reset and response** 1) Normal_mode_request resets TEC and REC to zero. 2) FCE shall assert Normal_mode_response to the LLC after servicing a Normal_mode_request.

ID	ISO ref	Priority	Requirement
REQ-032	8.1.4.2 rule a), rule b), rule c), rule d), rule e), rule f), rule g), rule h)	P1	**Error counter rules** 1) REC +1 on any receiver-detected error (except bit errors during error/overload flag). 2) REC +8 on first dominant bit after sending an error flag. 3) REC +8 on bit error while sending an active error or overload flag. 4) REC -1 on successful RX if in 1-127. Stays 0 if already 0. Set to 119-127 if above 127. 5) TEC +8 when sending an error flag (exceptions: error-passive ACK error, pre-arbitration stuff error). 6) TEC +8 on bit error while sending an active error or overload flag. 7) TEC -1 on successful TX (floor 0). 8) After 14 consecutive dominant bits following an active error or overload flag (8 after a passive error flag), increment both counters by 8. 9) Further +8 for each additional 8 consecutive dominant bits.
REQ-033	8.1.4.3, 8.1.4.4, 8.1.3.2	P1	**Error confinement state transitions** 1) Either counter exceeding 127 causes error-passive. 2) Both counters at or below 127 restores error-active. 3) TEC exceeding 255 causes bus-off. 4) Bus-off recovery requires 128 idle conditions; both counters reset to zero. 5) FCE shall assert Bus_off notification to the LLC on entering bus-off state.
REQ-034	6.6.11.3	P1	**BRS bit rate switching** 1) A recessive BRS bit switches from nominal to FD data rate at the BRS sample point. A dominant BRS bit keeps nominal rate. 2) The bit rate switches back to nominal at the sample point of the first CRC delimiter bit.
REQ-035	6.4.3, 6.4.4, Table 5	P1	**DLC byte-count mapping** 1) DLC 0-8 map linearly. FD extends DLC 9⟶12, 10⟶16, 11⟶20, 12⟶24, 13⟶32, 14⟶48, 15⟶64 bytes. CC treats DLC 9-15 as 8 bytes. 2) An LLC restricted to fewer bytes shall replace excess bytes with 0xCC padding in both TX and RX directions.
REQ-036	6.6.16	P3	**Bit transmission order** Within each field the MSB is transmitted first. Within the data field byte 0 is sent first. Within each byte bit(7) is sent first.

ID	ISO ref	Priority	Requirement
REQ-037	8.1.5	P2	**Node start-up** 1) A node shall complete bus re-integration (11 consecutive recessive bits) before transmitting. 2) A single node that receives no ACK shall become error-passive but shall never go bus-off. The error-passive ACK exception applies indefinitely.
REQ-038	6.6.18	P2	**MAC data consistency** Transmitted frames shall be protected against mid-transmission corruption by at least one of: (a) buffering the full frame before TX starts, or (b) LLC data-error checking with frame invalidation on detection.
REQ-039	6.6.21.1, 6.6.21.4	P2	**Error signalling enable** A configuration parameter shall exist to enable or disable error signalling. §6.6.21.4 defines the full behaviour when error signalling is disabled.

A.2 Verification Plan Fields

The verification plan table contains the verification metadata fields added during the planning phase. Cross-reference to the requirements table above via the shared ID field.

Table 10: ISO 11898-1 verification plan.

ID	Layer	Side	Format	Observability	Method	Status	Label	File
REQ-001	LLC	both	CB, CE, FB, FE	black_box	simulation	not_started		
REQ-002	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started		
REQ-003	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started		
REQ-004	LLC	receiver	CB, CE, FB, FE	white_box	code_inspection	not_started		
REQ-005	LLC	transmitter	CB, CE, FB, FE	black_box	simulation	not_started		
REQ-006	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	in_progress	p_output_vc, p_reset_checker	can_mac_crc_tb.vhd
REQ-007	system	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~900, test_bus_off	can_mac_pcs_fce_tb.vhd
REQ-008	MAC	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~486, test_normal, test_bus_off	can_mac_pcs_fce_tb.vhd
REQ-009	system	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~433, test_bus_off	can_mac_pcs_fce_tb.vhd
REQ-010	MAC	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~451, test_normal	can_mac_pcs_fce_tb.vhd
REQ-011	MAC	both	CB, CE	black_box	simulation	not_started		
REQ-012	MAC	both	CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~546, p_fsm:~548, test_normal	can_mac_pcs_fce_tb.vhd
REQ-013	MAC	both	CB, CE, FB, FE	white_box	code_inspection	in_progress	p_fsm:~299, p_fsm:~216	can_mac_fsm.vhd
REQ-014	system	both	CB, CE, FB, FE	white_box	simulation, waveform_inspection	in_progress	test_normal	can_mac_pcs_fce_tb.vhd

ID	Layer	Side	Format	Observability	Method	Status	Label	File
REQ-015	MAC	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~613, test_normal	can_mac_pcs_fce_tb.vhd
REQ-016	MAC	transmitter	FB, FE	white_box	simulation, waveform_inspection	in_progress	test_normal	can_mac_pcs_fce_tb.vhd
REQ-017	MAC	both	FB, FE	black_box	simulation	in_progress	p_sbc_checker	can_mac_bs_tb.vhd
REQ-018	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~768, p_fsm:~796, test_normal	can_mac_pcs_fce_tb.vhd
REQ-019	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	in_progress	p_stuff_bit_checker, p_fsb_checker	can_mac_bs_tb.vhd
REQ-020	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	in_progress	test_normal	can_mac_pcs_fce_tb.vhd
REQ-021	system	transmitter	CB, CE, FB, FE	black_box	simulation	in_progress	test_lost_arb	can_mac_pcs_fce_tb.vhd
REQ-022	MAC	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~283, p_fsm:~336, p_fsm:~343, p_fsm:~415, test_bus_off	can_mac_pcs_fce_tb.vhd
REQ-023	MAC	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~283, p_fsm:~294, p_fsm:~357, test_bus_off	can_mac_pcs_fce_tb.vhd
REQ-024	MAC	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~370	can_mac_fsm.vhd
REQ-025	PCS	both	CB, CE, FB, FE	white_box	simulation	in_progress	test_normal	can_pcs_tb.vhd
REQ-026	system	both	CB, CE, FB, FE	black_box	simulation, coverage	in_progress	test_delay_sweep	can_mac_pcs_fce_tb.vhd
REQ-027	PCS	transmitter	FB, FE	black_box	simulation	in_progress	p_check_tdc_delay	can_pcs_tb.vhd
REQ-028	PCS	both	CB, CE, FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_can_pcs:~177, p_can_pcs:~228, test_normal	can_pcs_tb.vhd
REQ-029	PCS	both	CB, CE, FB, FE	black_box	simulation	in_progress	test_normal	can_pcs_tb.vhd

ID	Layer	Side	Format	Observability	Method	Status	Label	File
REQ-030	system	both	CB, CE, FB, FE	white_box	simulation, code_inspection	in_progress	test_bus_off	can_mac_pcs_fce_tb.vhd
REQ-031	FCE	both	CB, CE, FB, FE	black_box	simulation	in_progress	test_reset, test_bus_off_recovery	can_fce_tb.vhd
REQ-032	FCE	both	CB, CE, FB, FE	black_box	simulation	in_progress	test_rule_a, test_rule_b, test_rule_c, test_rule_c_exception, test_rule_d, test_rule_e, test_rule_f_tx, test_rule_f_rx, test_rule_g, test_rule_h	can_fce_tb.vhd
REQ-033	FCE	both	CB, CE, FB, FE	black_box	simulation	in_progress	test_bus_off, test_bus_off_recovery	can_fce_tb.vhd
REQ-034	MAC	both	FB, FE	white_box	code_inspection, waveform_inspection	in_progress	p_fsm:~636, p_fsm:~755, test_normal	can_mac_pcs_fce_tb.vhd
REQ-035	LLC	both	CB, CE, FB, FE	black_box	simulation, coverage	not_started		
REQ-036	MAC	both	CB, CE, FB, FE	black_box	simulation	not_started		
REQ-037	system	both	CB, CE, FB, FE	black_box	simulation	not_started		
REQ-038	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started		
REQ-039	MAC	both	CB, CE, FB, FE	white_box	code_inspection	waived		

[1] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer*. 2024.

[2] M. Jeřábek and P. Píša, "CTU CAN FD — open-source CAN FD IP core." 2019. Available: https://gitlab.fel.cvut.cz/canbus/ctucanfd_ip_core

[3] M. Jeřábek, "Open-source and open-hardware CAN FD protocol support," PhD thesis, Czech Technical University in Prague, 2019. Available: https://dspace.cvut.cz/handle/10467/82630

[4] International Organization for Standardization, *Road vehicles — controller area network (CAN) conformance test plan — Part 1: Data link layer and physical signalling*. 2016.

[5] OpenCores Contributors, "CAN protocol controller — OpenCores." 2003. Available: https://opencores.org/projects/can

[6] S. V. Nøsen, "Canola — CAN controller in VHDL." 2020. Available: https://github.com/svnesbo/canola

[7] Robert Bosch GmbH, "M_CAN — CAN FD protocol controller IP module." 2024. Available: https://www.bosch-semiconductors.com/ip-modules/can-fd/

[8] F. Hartwich, "CAN with flexible data-rate," in *Proceedings of the 13th international CAN conference (iCC 2012)*, 2012.

[9] AMD (Xilinx), "CAN FD controller — Vivado design suite product guide (PG223)." 2024. Available: https://docs.amd.com/r/en-US/pg223-canfd

[10] CAST, Inc., "CAN-FD protocol controller." 2024. Available: https://www.cast-inc.com/automotive/can/can-fd-protocol-controller

[11] GHDL Contributors, "GHDL — open-source VHDL simulator." 2024. Available: https://ghdl.github.io/ghdl/

[12] OSVVM Contributors, "OSVVM — open source VHDL verification methodology." 2024. Available: https://osvvm.org

[13] Intel Corporation, "Avalon interface specifications," Intel Corporation, 683091, 2021. Available: https://www.intel.com/programmable/technical-pdfs/683091.pdf

[14] J. Bergeron, "Verification planning," in *Writing testbenches: Functional verification of HDL models*, 2nd ed., Kluwer Academic Publishers, 2003.